COURSE CODE

SBT-DF204

COURSE TITLE

Computer Forensics Case Studies

LAB TITLE

Investigating Rhion Possession Evidence

NAME

Kumpon Emmanuel Nandom

STUDENT ID

2025/FWSD/11325

DATE

26/10/2026

PLATFORM USED

Kali Linux

INSTRUCTOR

Mr. Aminu Idris, AMCPN

Commandant

Executive conclusion

The USB image contains files and shows a series of network transfers. However, it does not indicate who used any device. RHINOUSB.dd is a FAT16 volume with no partition table. It holds two recipe files, and there are no deleted directory entries. Using PhotoRec 7.2, I recovered nine image files—seven in JPEG format and two in GIF format. Eight of these files are unique based on their SHA-256 values.

In rhino.log, a client with the IP address 137.30.122.253 uploaded four files to the FTP server at 137.30.120.40 on April 26, 2004, between 22:21 and 22:27 UTC. The last file uploaded was an encrypted archive named contraband. zip. I managed to reconstruct it, confirmed it was complete, found its password using a dictionary in the lab, and extracted a file called rhino2.jpg. This file has the same size as f0106409.jpg, which I carved from the USB image (230,665 bytes), and both images show the same picture.

In rhino2.log, a client at 137.30.123.234 requested rhino4.jpg and rhino5.gif from 137.30.120.37 on April 28, 2004, at 21:08 UTC. Rhinos3.log includes a response to a request for rhino.exe around 21:16 UTC. The client's Ethernet address, 00:03:93:cc:57:92, appears in both the FTP and HTTP captures.

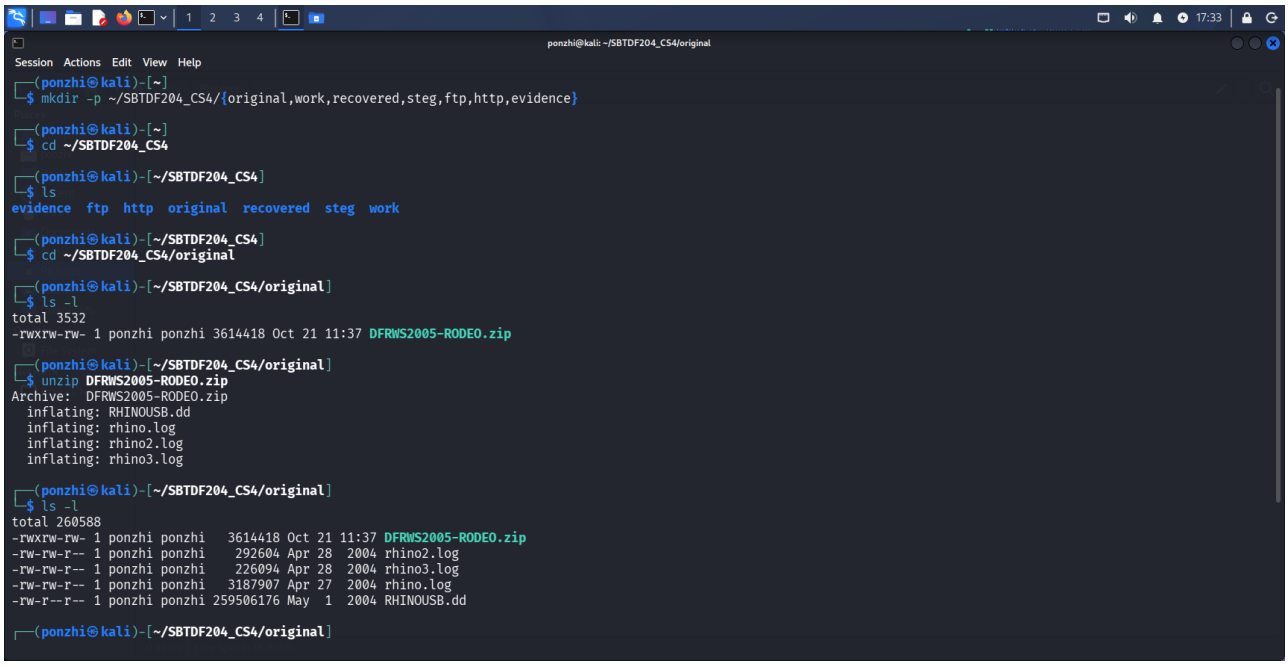
I could not complete a steganography test because stegdetect would not build on this version of Kali. Therefore, this report does not conclude anything about hidden content. I also could not reconstruct two FTP uploads: rhino1.jpg and rhino3.jpg. There is a moderate level of confidence that one network card produced both captures, low to moderate confidence that it is linked to the USB image, and no confidence regarding the identity of any person.

Evidence acquisition and integrity

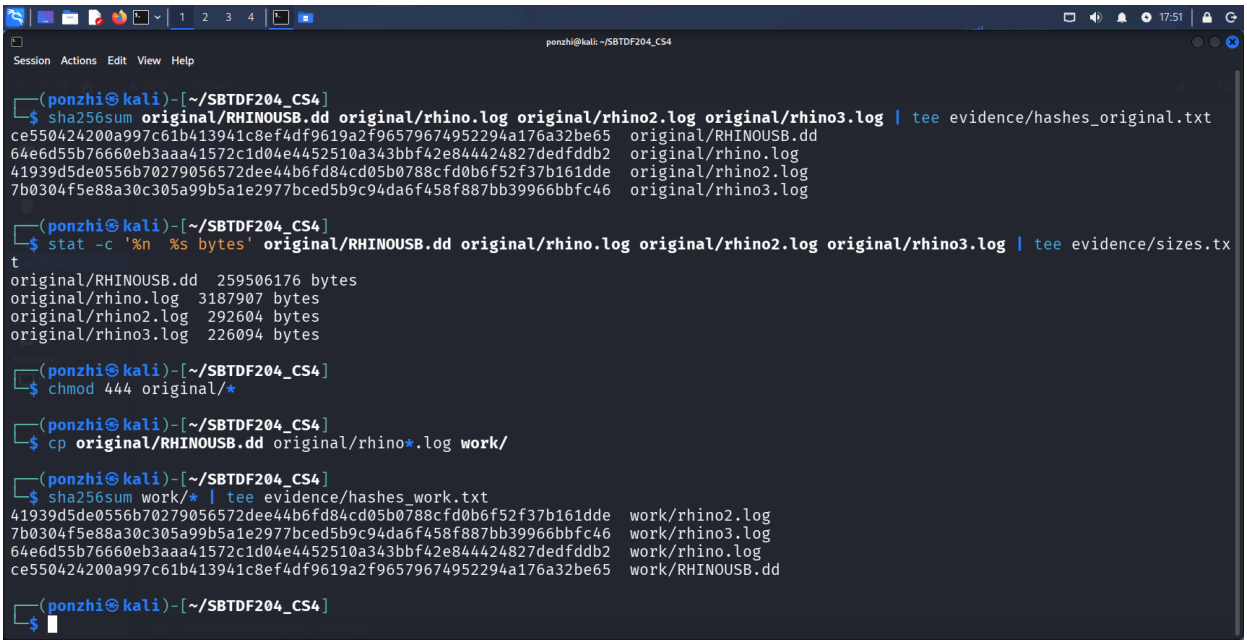
The package DFRWS2005-RODEO.zip was placed in ~/SBTDF204_CS4/original and unzipped there. Its four members are the originals for this case.

Original	Bytes	SHA-256	Working copy
RHINOUSB.dd	259,506,176	ce550424200a997c61b413941c8ef4df9619a2f96579674952294a176a32be65	work/RHINOUSB.dd and work/RHINOUSB_photorec.dd
rhino.log	3,187,907	64e6d55b76660eb3aaa41572c1d04e4452510a343bbf42e844424827dedfddb2	work/rhino.log, copy in ftp/
rhino2.log	292,604	41939d5de0556b70279056572dee44b6fd84cd05b0788cfd0b6f52f37b161dde	work/rhino2.log, copy in http/
rhino3.log	226,094	7b0304f5e88a30c305a99b5a1e2977bced5b9c94da6f458f887bb39966bbfc46	work/rhino3.log, copy in http/

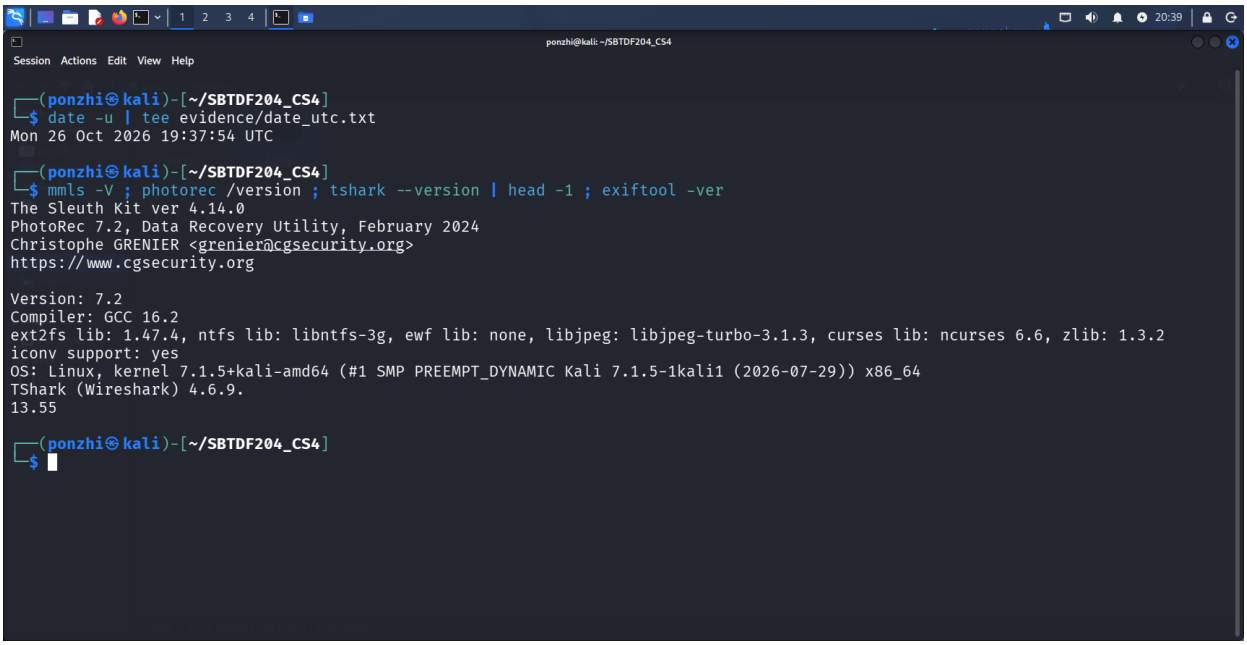
I noted the size of each original file with the command "stat" and calculated their SHA-256 hashes using "sha256sum." I made the original files read-only with "chmod 444" and then copied them into a folder called "work/." I ran "sha256sum" again on the copies, and I got the same four hash values. I did not hash the zip file itself, so the hashes refer to the files extracted from it.
I recorded the reference time as Mon 26 Oct 2026 19:37:54 UTC. All work was done under one local user account. The original files remained read-only, and I ran every command on the copies. I used the internet to download tools and a public wordlist called "rockyou.txt," which is 139,921,497 bytes.



Evidence package unzipped in original/, with sizes and archive timestamps of the four members. (E01)



Byte size and SHA-256 of each original, chmod 444 on the originals, copy to work/, and SHA-256 of the working copies. (E01, E02)



UTC reference time from date -u and tool versions: Sleuth Kit 4.14.0, PhotoRec 7.2, TShark 4.6.9, ExifTool 13.55. (E03)

Method

Disk Image: I identified the disk image using the 'file' command and 'fdisk -l'. I checked for partitions using 'mmls', examined the file system layout with 'fsstat', and listed both allocated and deleted files using 'fls -r' and 'fls -rd'. I accessed two allocated files with 'icat'. There was no need for an offset option because 'mmls' did not find a partition table.

Recovery: I made a second copy of the working image and verified it matched with 'sha256sum'. I then ran PhotoRec 7.2 on the copy, saving the recovered files in a folder named 'recovered/'. I identified each carved file using 'file', checked their hashes with 'sha256sum', viewed their sizes with 'ls -l', and opened images using ImageMagick 'display'. I confirmed the uniqueness of files by checking their SHA-256 values.

Hidden Content: I moved the carved JPEG files to a folder called 'steg/' and checked their metadata with ExifTool. I attempted to build 'stegdetect' from source to test for embedded data. If it found any, I planned to pass the files to 'stegbreak' and 'jpseek'.

FTP: I opened a copy of 'rhino.log' in Wireshark and filtered using 'ftp.request.command == "STOR" || ftp.request.command == "RETR"' and 'ftp.response.code == 150' to find each transfer. For each object, I used 'Follow TCP Stream', set 'Show data as' to 'Raw', and saved the bytes. I checked the saved files using 'xxd' (for the first and last bytes), 'file', 'unzip -l', and 'sha256sum'. For the encrypted archive, I used 'fcrackzip' with 'rockyou.txt', but only on this course archive.

HTTP: I examined 'rhino2.log' and 'rhino3.log' in Wireshark. For 'rhino2.log', I filtered with 'http.request.method == "GET"' and exported all objects to a folder named 'http/'. For 'rhino3.log', I filtered with 'tcp.stream eq 3', used 'Follow HTTP Stream' with 'Show data as' set to 'Raw', and saved the conversation. I validated the objects using 'file', 'xxd', 'sha256sum', and 'display'. The executable was never run.

Time Handling: In the screenshots from 'rhino2.log' and 'rhino3.log', Wireshark displayed time in UTC with a 'Z' suffix. In the screenshots from 'rhino.log', it showed local time without a suffix. The Kali clock is one hour ahead of UTC (the file times in Figure 22 show +01:00), so I subtracted one hour from the 'rhino.log' times. All times in the timeline are in UTC. Frame numbers refer to Wireshark display numbers in the unmodified captures.

Findings

Question 1. How did I preserve the evidence?

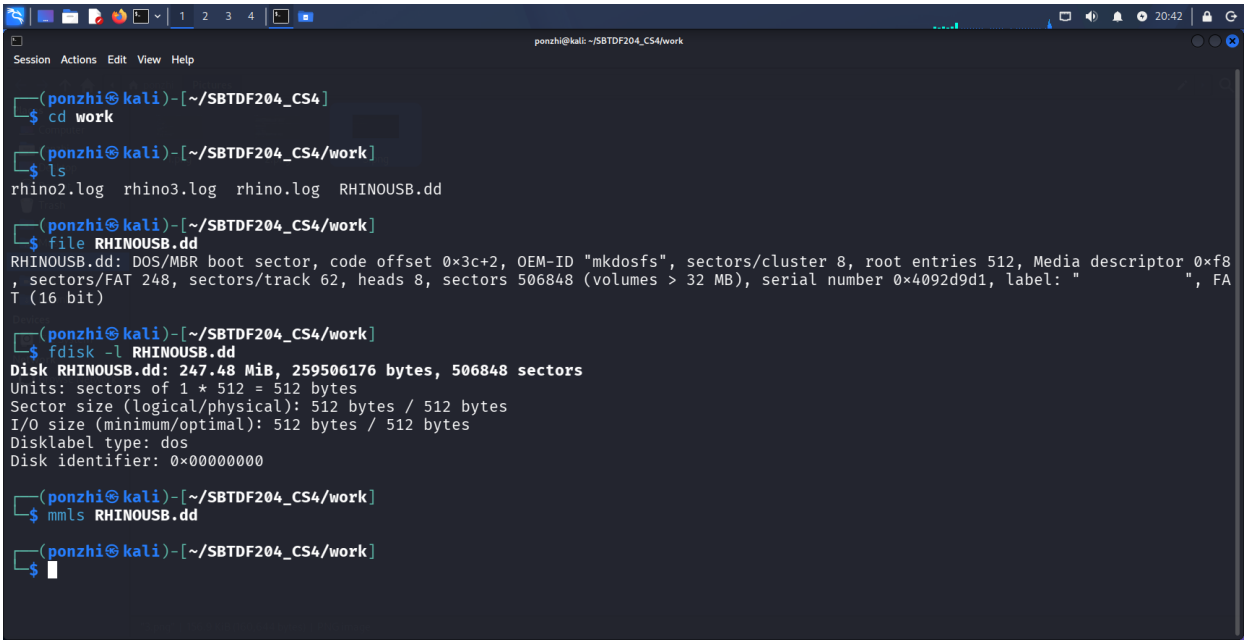
The steps to preserve the files, their sizes, and hashes are in E01 to E03. The original files are set to read-only, and the working copies have the same SHA-256 values. I also created a separate copy of the image for PhotoRec and checked it for accuracy before using it.

Question 2. What does the USB image contain before any recovery?

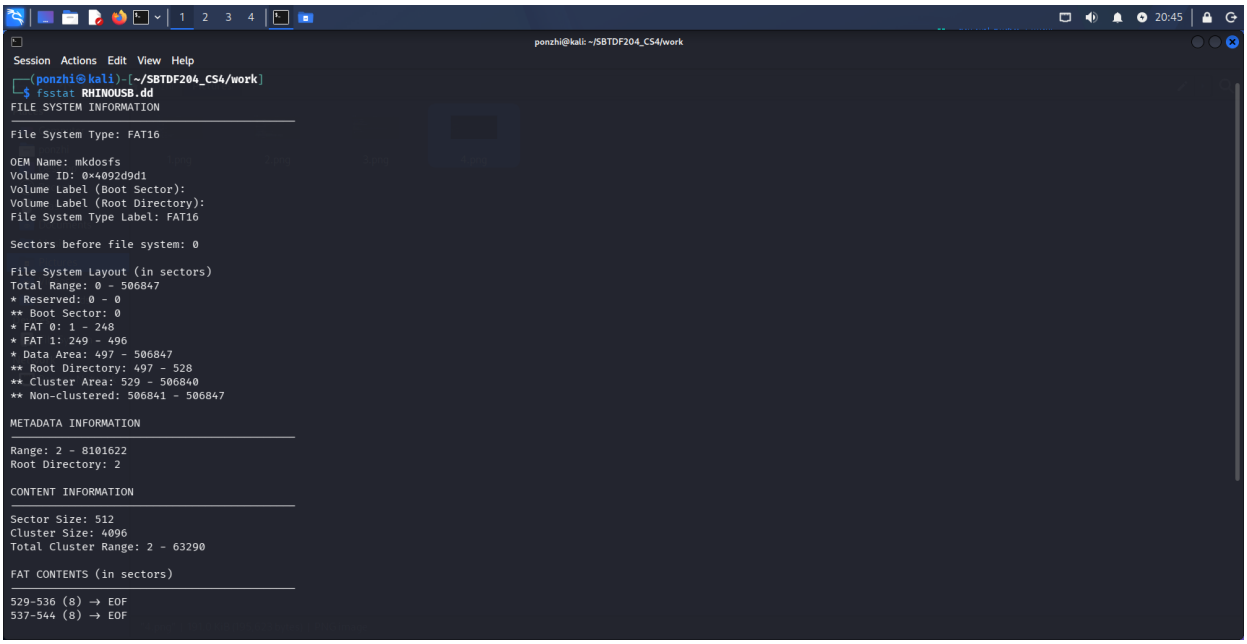
The file RHINOUSB.dd is recognized as a DOS/MBR boot sector. It has an OEM-ID of mkdosfs, with 8 sectors per cluster and 512 root entries. The media descriptor is 0xf8, and the file has 248 sectors per FAT, totaling 506,848 sectors. Its volume serial is 0x4092d9d1, and it uses a 16-bit FAT. According to the fdisk -l command, the disk is 247.48 MiB in size, or 259,506,176 bytes, with 512-byte sectors. It has a DOS label type and a disk identifier of 0x00000000. The mmls command did not show any rows, indicating the image has one file system starting at sector 0 and no partition.

The fsstat command identified the file system as FAT16. The two FAT copies are located at sectors 1 to 248 and 249 to 496. The root directory is at sectors 497 to 528, while the cluster area is at sectors 529 to 506,840, using 4,096-byte clusters. The first two FAT chains, located at sectors 529 to 536 and 537 to 544, each end after one cluster.

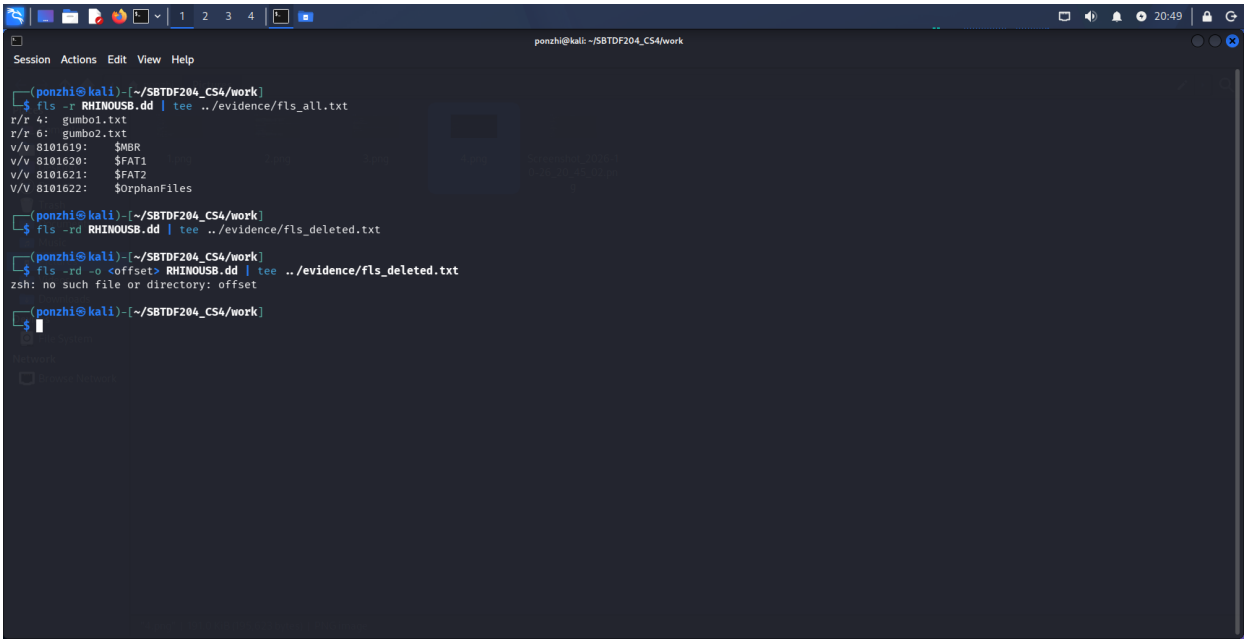
Using fls -r, we found two regular files: gumbo1.txt at inode 4 and gumbo2.txt at inode 6. There are also virtual entries for $MBR, $FAT1, $FAT2, and $OrphanFiles. The fls -rd command returned nothing, which means there are no deleted directory entries or orphan entries.
When we used icat on inodes 4 and 6, we retrieved two cooking recipes: one for shrimp and tasso gumbo and another for shrimp and andouille gumbo. These texts do not relate to the accusation.
The directory structure does not provide any details about the image. The images exist only as recovered data. I did not record FAT timestamps with istat, so the file system does not provide any dated events for the timeline.



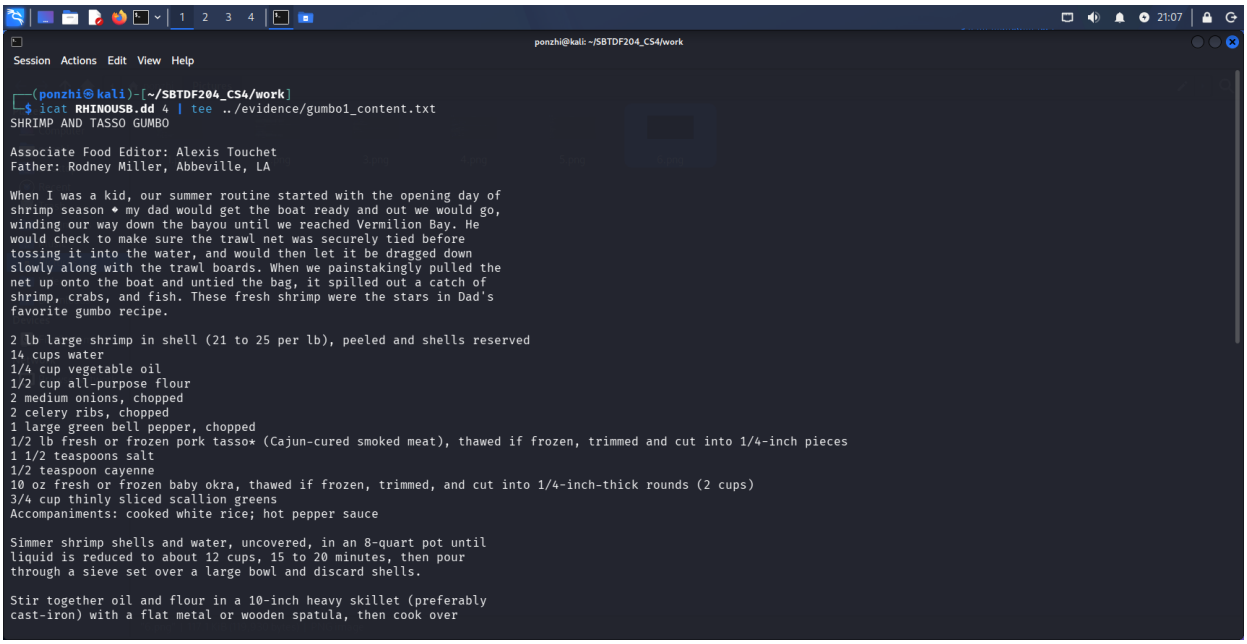
file, fdisk -l and mmls on the working copy of RHINOUSB.dd. mmls returns no rows. (E04)



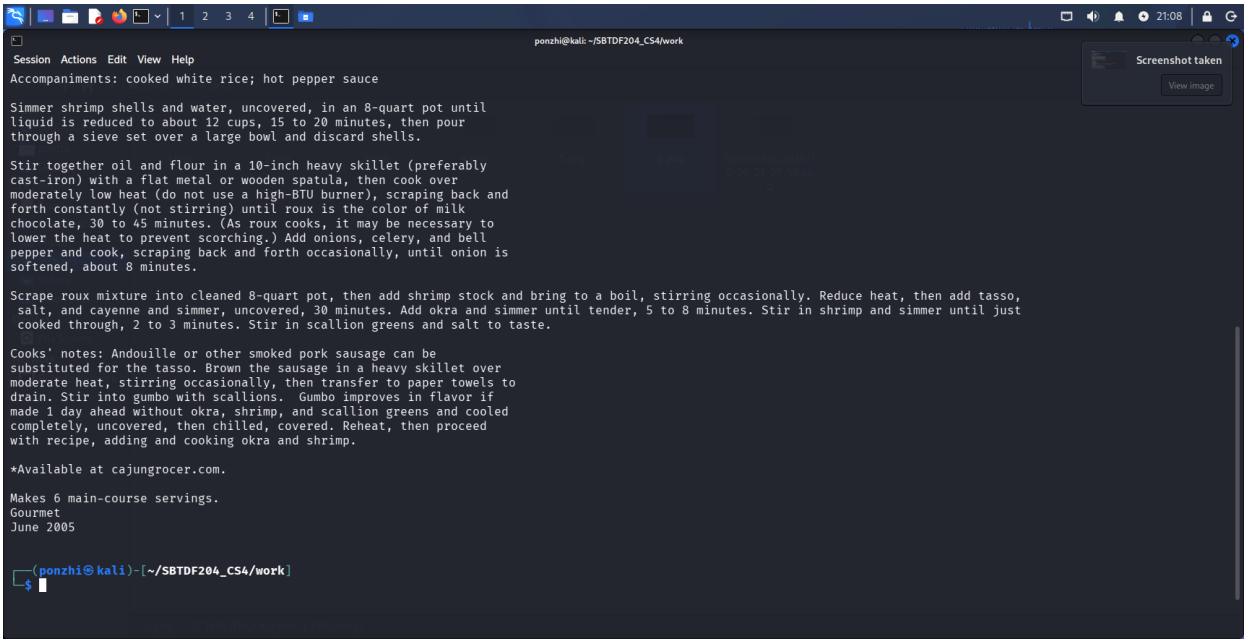
fsstat output for RHINOUSB.dd: FAT16 layout and cluster size. (E05)



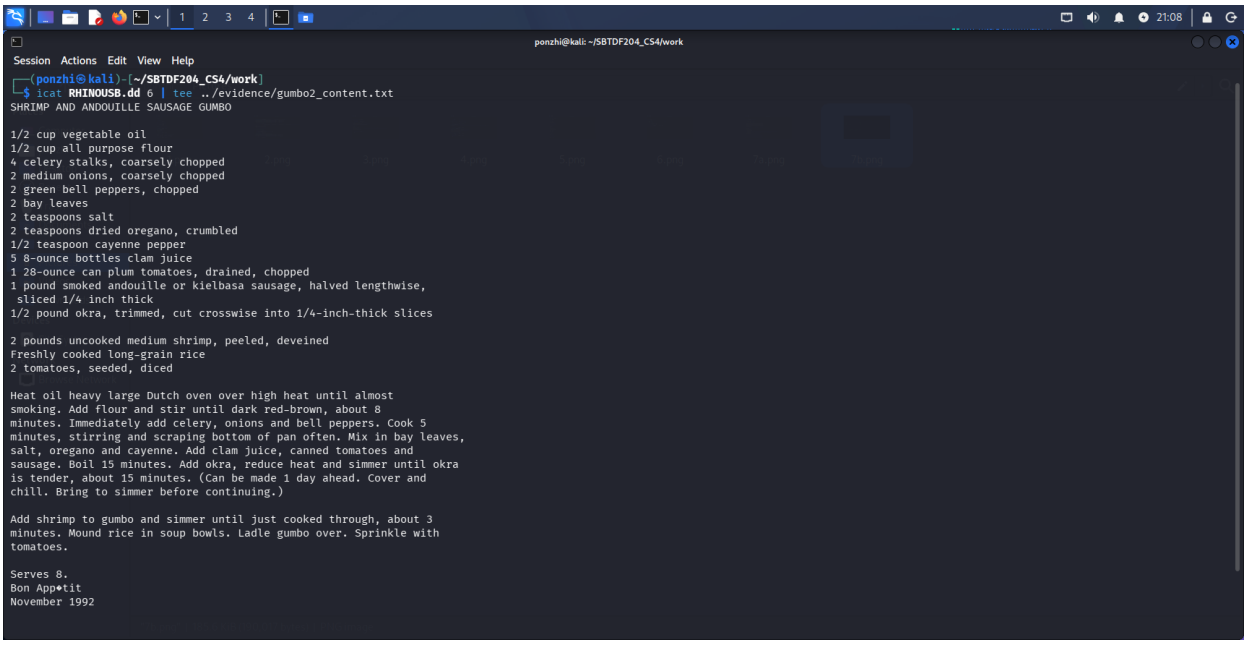
fls -r and fls -rd: two allocated files and no deleted entries. The last command contains an unused placeholder offset. (E06)



icat of inode 4 (gumbo1.txt), first part. (E07)



icat of inode 4 (gumbo1.txt), end of the text. (E07)



icat of inode 6 (gumbo2.txt). (E07)

Question 3. What did recovery produce?

PhotoRec 7.2 ran on the file RHINOUSB_photorec.dd, which is an exact copy of the original file. It found one FAT16 partition with 506,848 sectors that starts at sector 0 and saved 132 files in the recovered/recup_dir.1 folder. The recovery scope menu option did not appear in the capture.

The program identified the recovered files as seven JPEG images, two GIF images, one Word document, PhotoRec's report.xml, and some text fragments in ASCII or ISO-8859 format. Table 2 lists the nine image files. PhotoRec names the files based on the sector where they start. For example, f0104057.jpg starts at sector 104057 and f0335081.jpg starts at sector 335081. No original names, paths, or timestamps from the directory entries remain.

Image files carved from RHINOUSB_photorec.dd

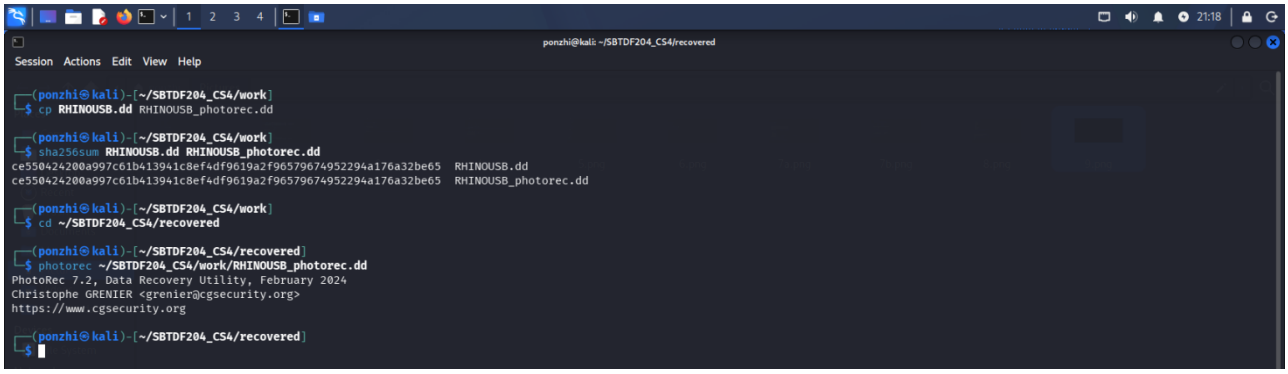
File	Type and bytes	SHA-256 (first 16)	Visual check
f0104057.jpg	JPEG, 95,814	2261020a0b31fe10	Opened, not captured
f0104249.jpg	JPEG, 415,534	8cdb89c6778d3409	Not opened in the captured session
f0105065.jpg	JPEG, 411,361	452075313c93ae31	Opened, not captured
f0105873.jpg	JPEG, 264,600	f92654d9ee17ab6b	Opened, not captured. Same hash as f0335081.jpg
f0106393.jpg	JPEG, 6,809	568457ed594c4e98	Rhino photograph
f0106409.jpg	JPEG, 230,665	b4f6bbb8d846d650	Rhino with a calf at water
f0106865.gif	GIF, 11,407	8a67d406ed130c9b	Rhino drawing

File	Type and bytes	SHA-256 (first 16)	Visual check
f0106889.gif	GIF, 4,105	71e9f6b94e0496cc	Blue illustrated rhino
f0335081.jpg	JPEG, 264,600	f92654d9ee17ab6b	Opened, not captured. Duplicate of f0105873.jpg

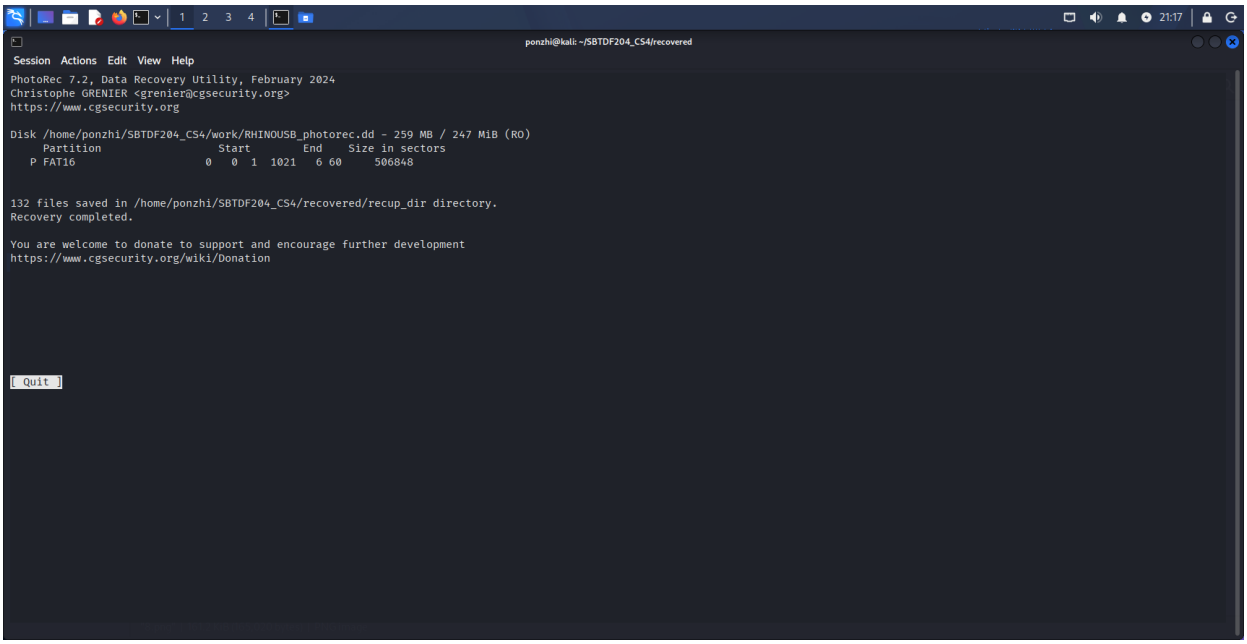
f0105873.jpg and f0335081.jpg have the same size and the same SHA-256 value, so nine files carry eight unique images. My captures show four of the nine files on screen (E13). The other images were opened with Display, but the image window is not in a capture, so I describe them by type, size and hash only.

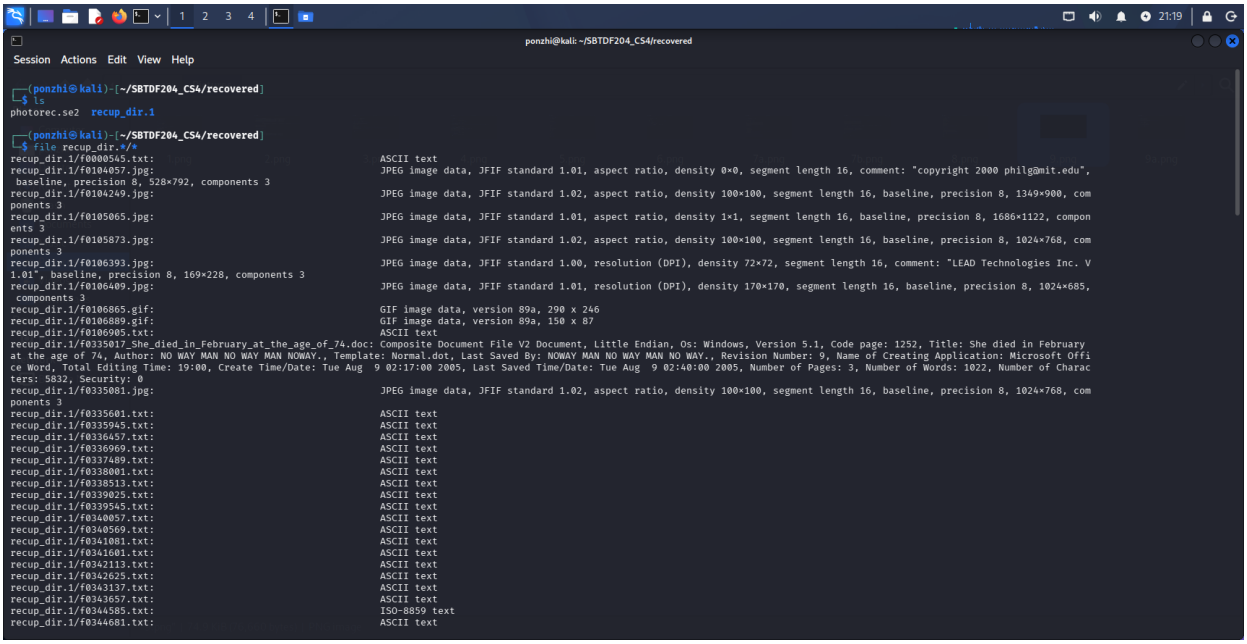
The carved Word document f0335017_She_died_in_February_at_the_age_of_74.doc has 3 pages and 1,022 words, with author metadata and creation and save times in August 2005. I recorded it for completeness and did not examine its text.

Carving shows that the data existed in the image. It does not show when the files were written or deleted, who opened them, or whether they were ever displayed on the device.

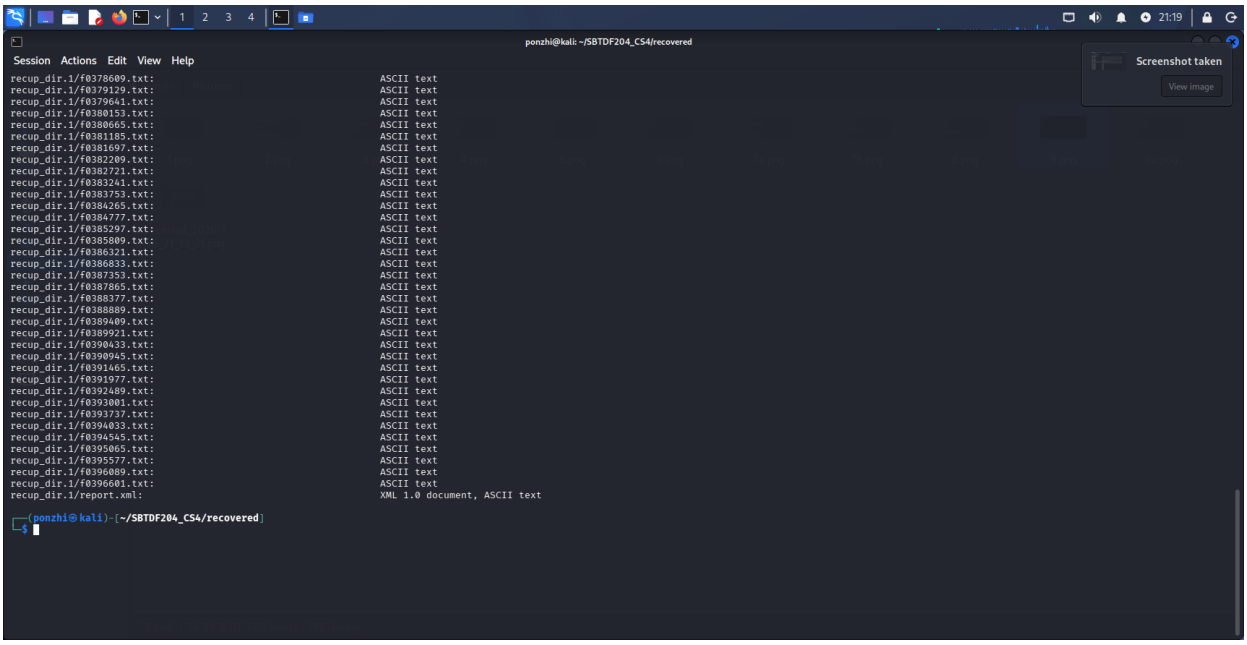


Working copy made for PhotoRec, SHA-256 equal to the original, and the PhotoRec command. (E08)

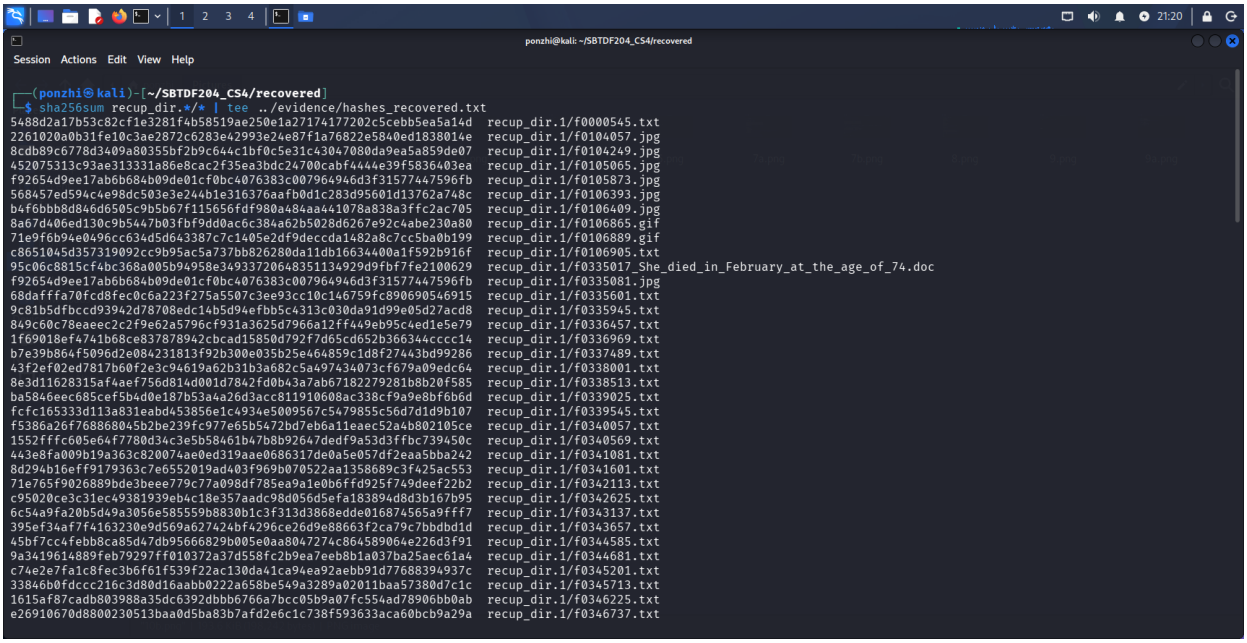




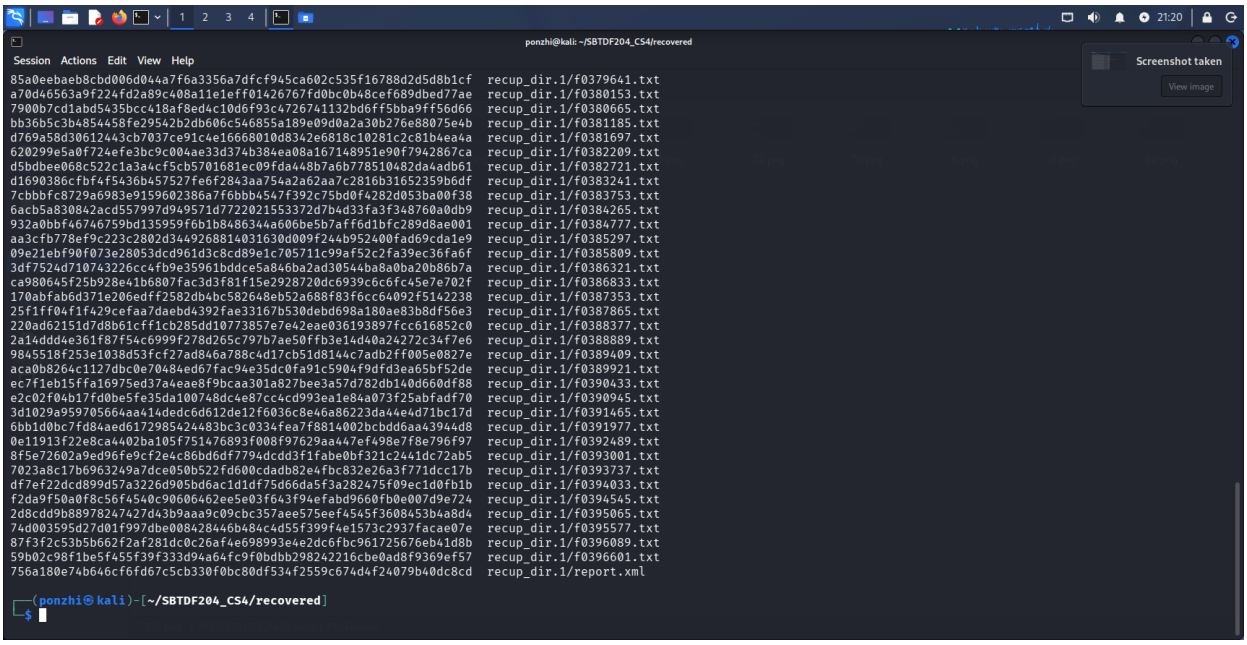
file output for the carved files, first part (JPEG, GIF and Word entries). (E10*)*



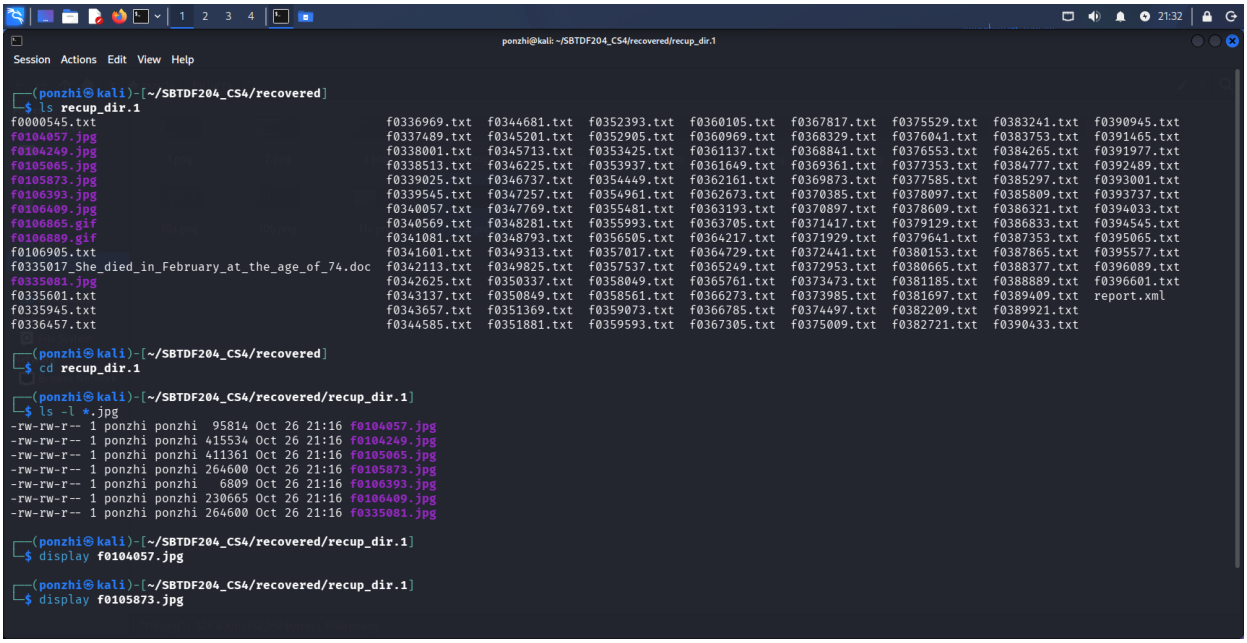
File output for the carved files. (E10)



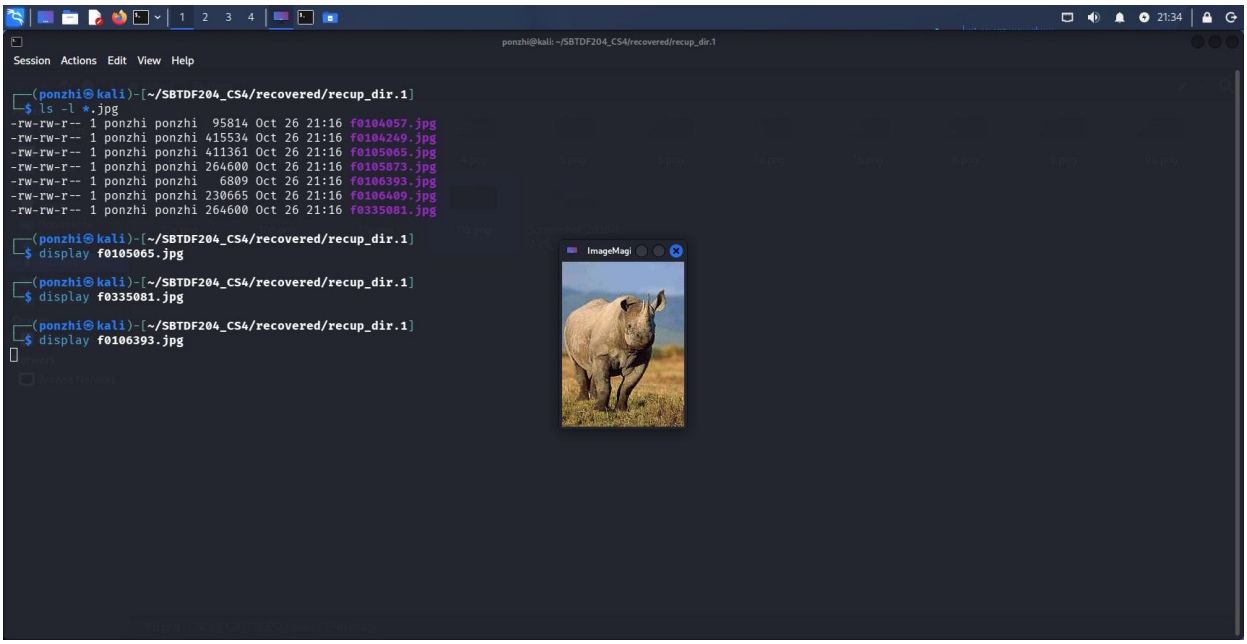
SHA-256 of the carved files, first part. f0105873.jpg and f0335081.jpg share one value. (E11)



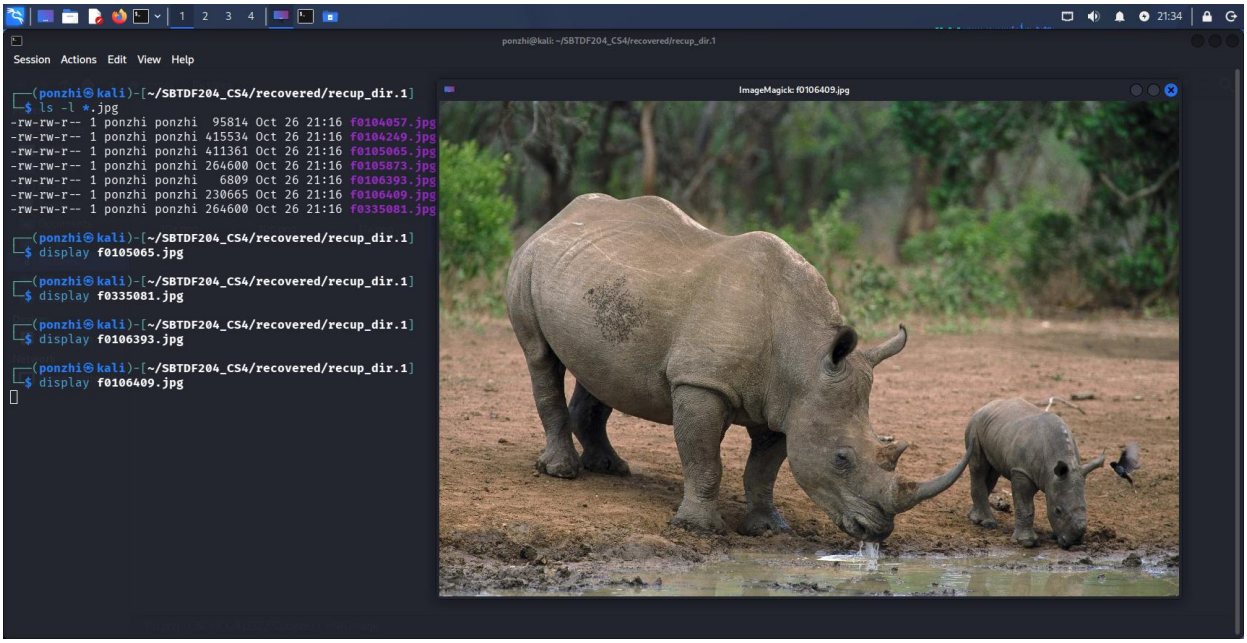
SHA-256 of the carved files, remaining text fragments. (E11)



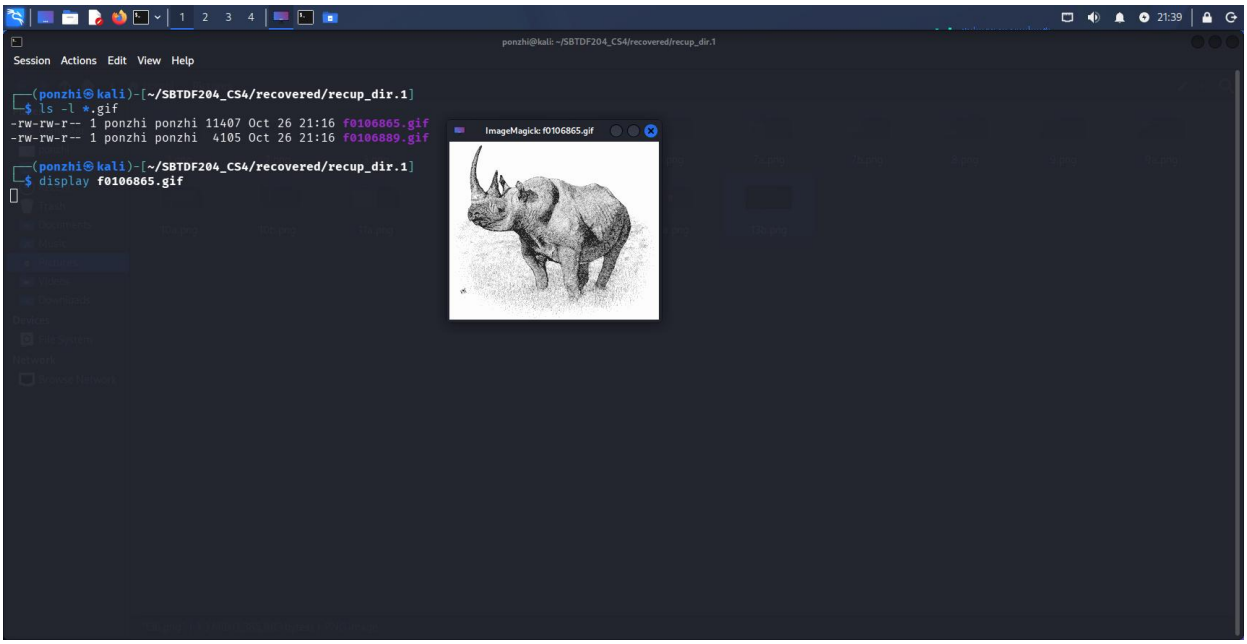
Contents of recup_dir.1 and sizes of the seven carved JPEG files. (E12)



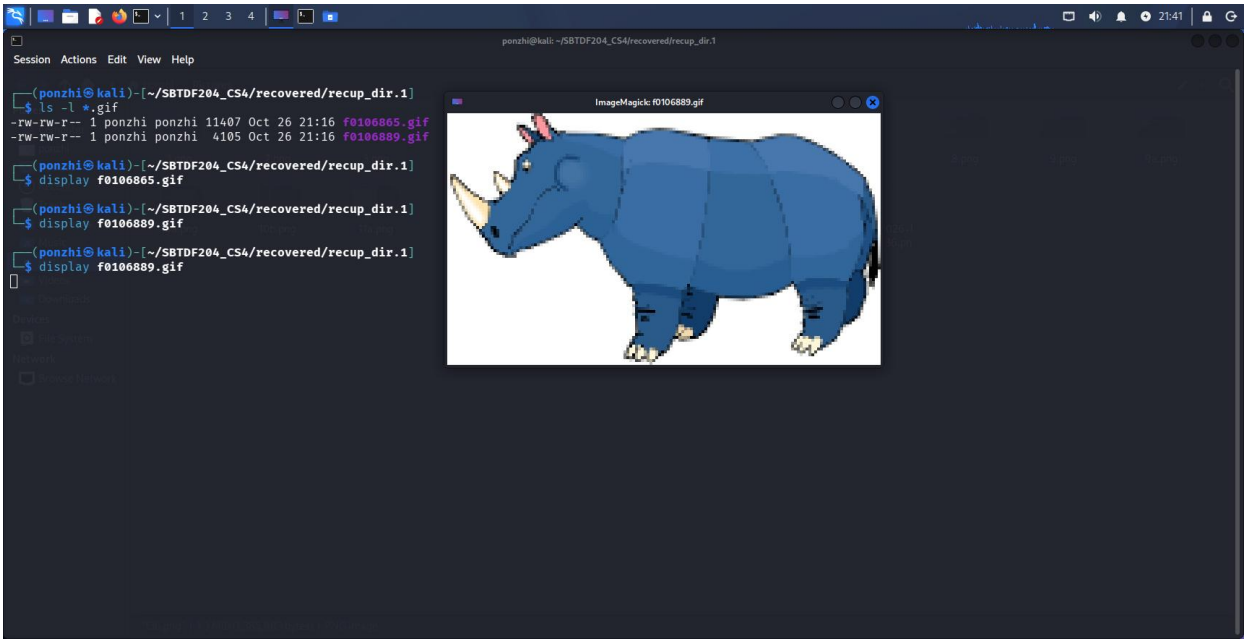
f0106393. (E13)



f0106409.jpg displayed. (E13)



Sizes of the two carved GIF files, and f0106865.gif displayed.

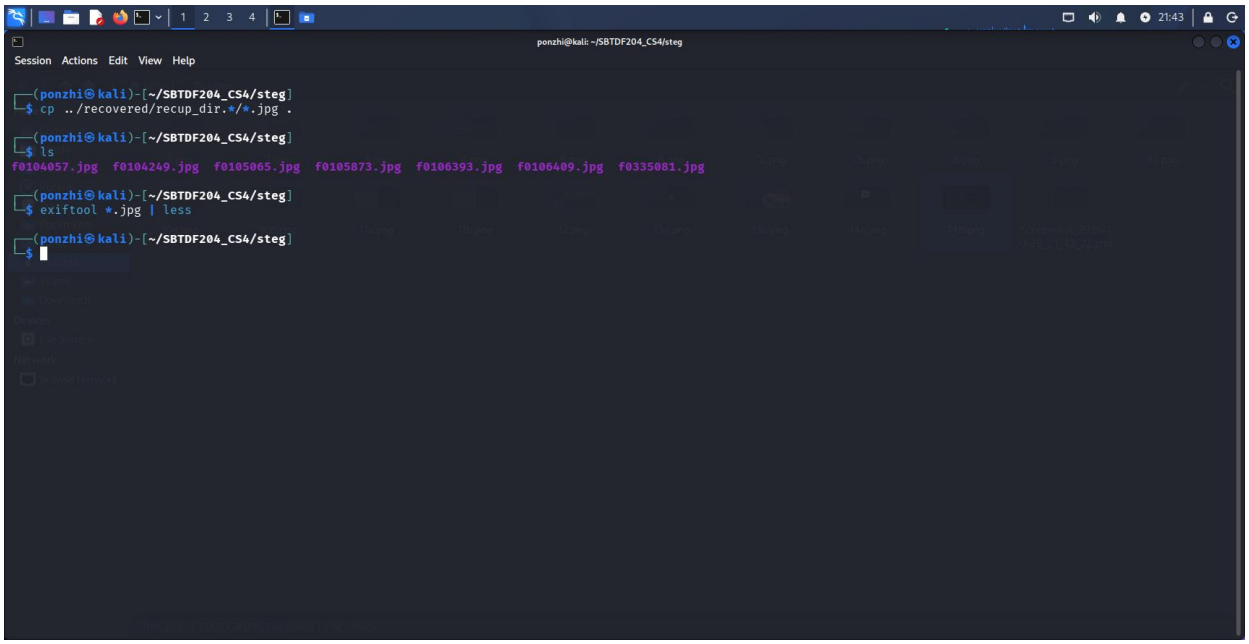


f0106889.gif displayed.

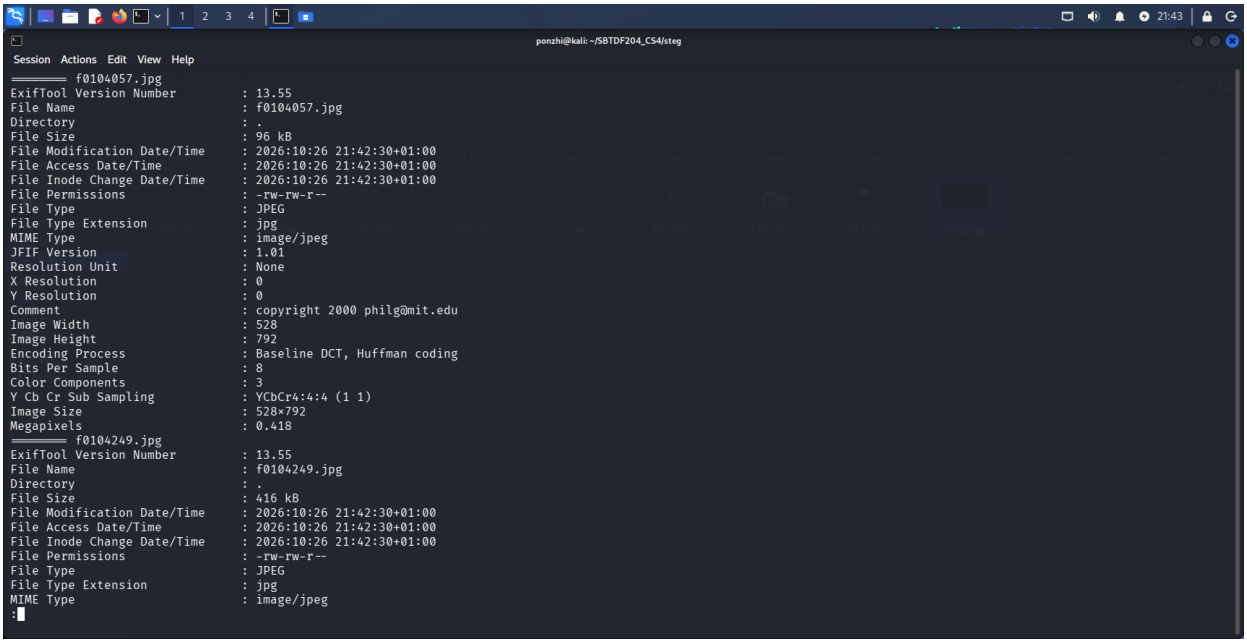
Question 4. What was found about hidden content?

I copied seven JPEG files into the folder named "steg/" (see Figure 21, E14) and used ExifTool to analyze them. The output included details from the files f0104057.jpg and the beginning of f0104249.jpg. One comment found was "copyright 2000 philg@mit.edu," and the image size is 528 x 792 pixels. Another file, f0106393.jpg, showed a comment "LEAD Technologies Inc. V1.01." There was no evidence of any added or hidden data in the captured fields.

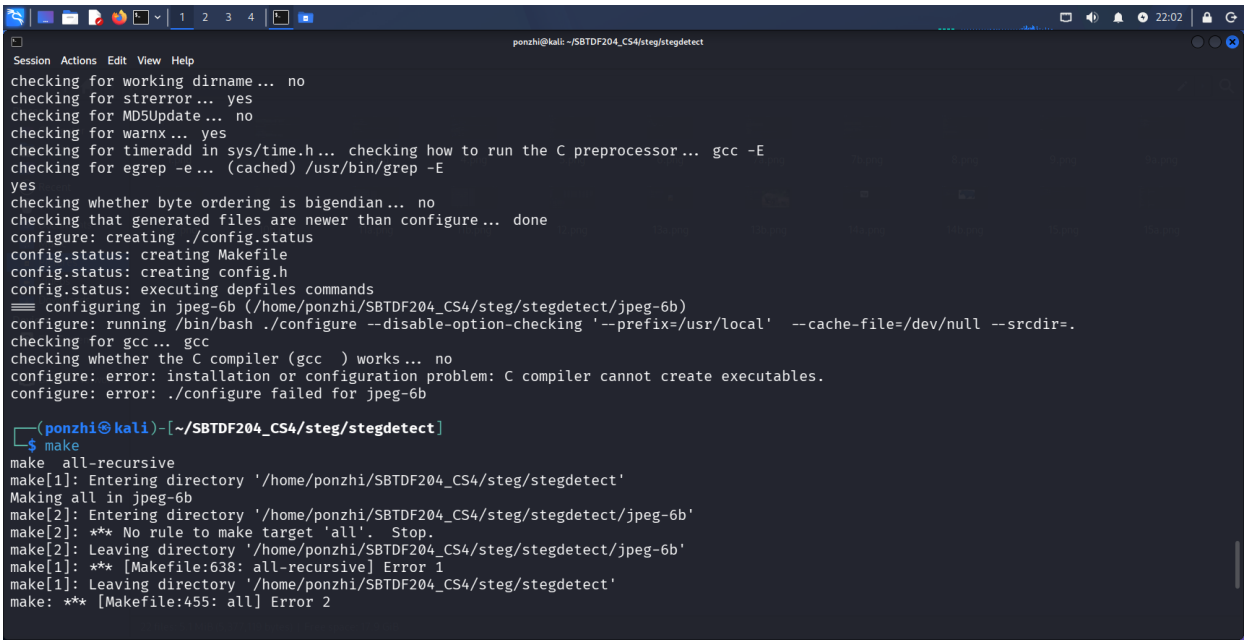
Next, I tried to build stegdetect from the source code. The initial configuration step ran successfully and created a Makefile and config.h file. However, the included jpeg-6b library did not pass its compiler test, giving the message "C compiler cannot create executables," and the make command failed with "No rule to make target 'all'."



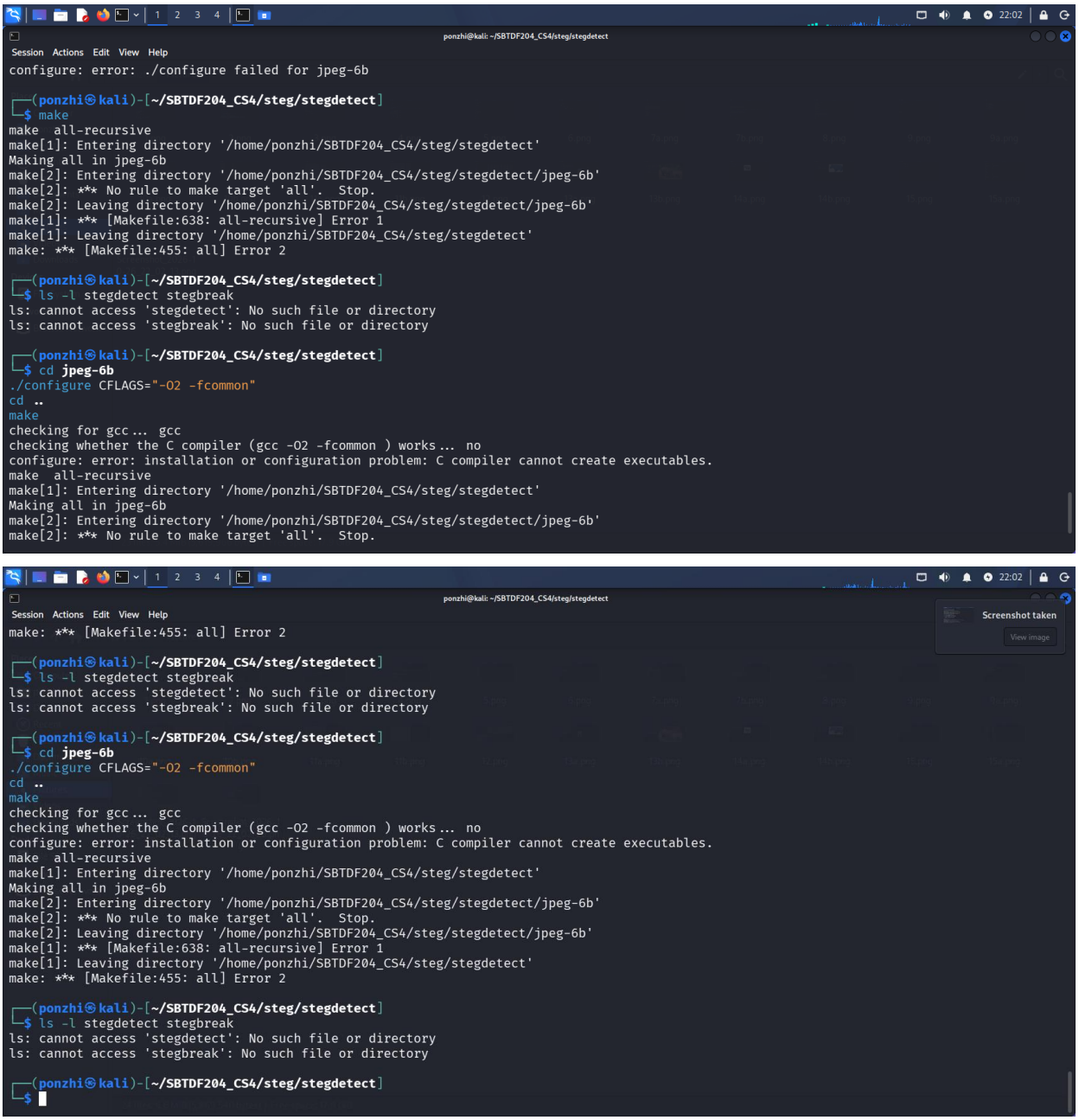
A search was run, and nothing was extracted. The evidence neither supports nor rules out hidden content in the JPEG files, and I make no claim either way.



ExifTool output for f0104057.jpg and the start of f0104249.jpg.



stegdetect build top level configure completes, jpeg-6b configure fails, make stops.



ls finds neither stegdetect nor stegbreak after the build attempts.

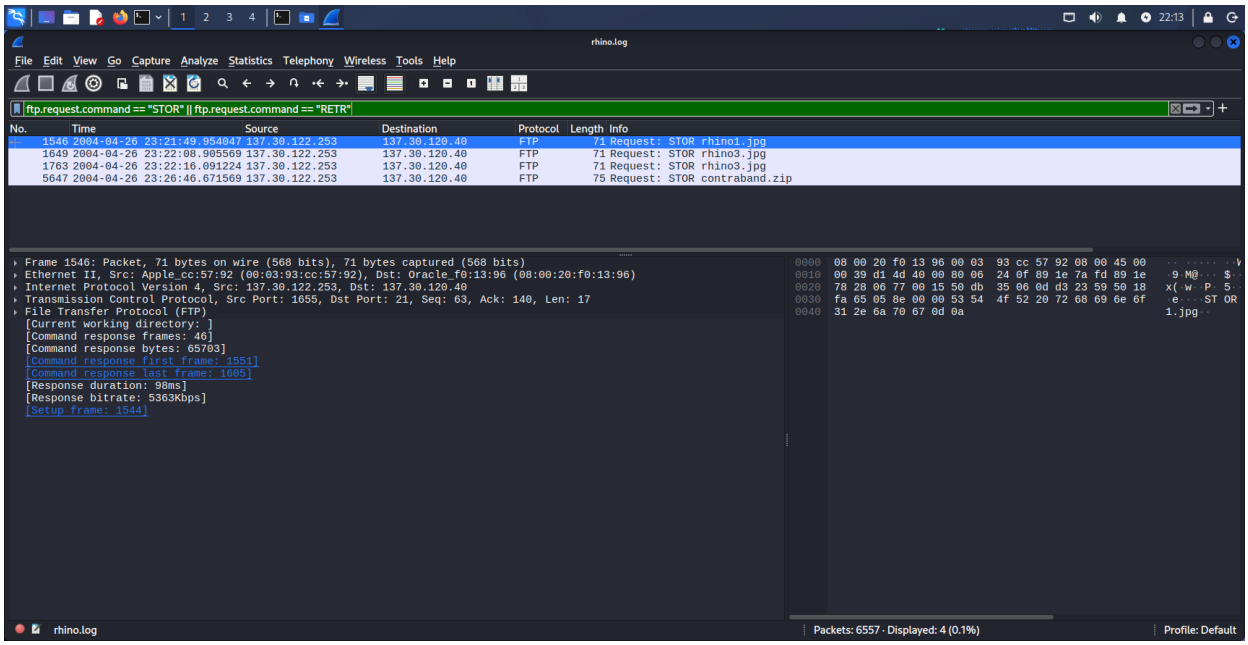
Question 5. What does the FTP capture show?

The file rhino.log has 6,557 packets. A filter for ftp.request.command showing "STOR" or "RETR" reveals four packets. All four are STOR commands sent from the IP address 137.30.122.253 to 137.30.120.40 on port 21. This means that each transfer in this log is an upload from the client to the server. Below is a table listing the times in UTC, along with the 150 responses that indicate when each data connection opens.

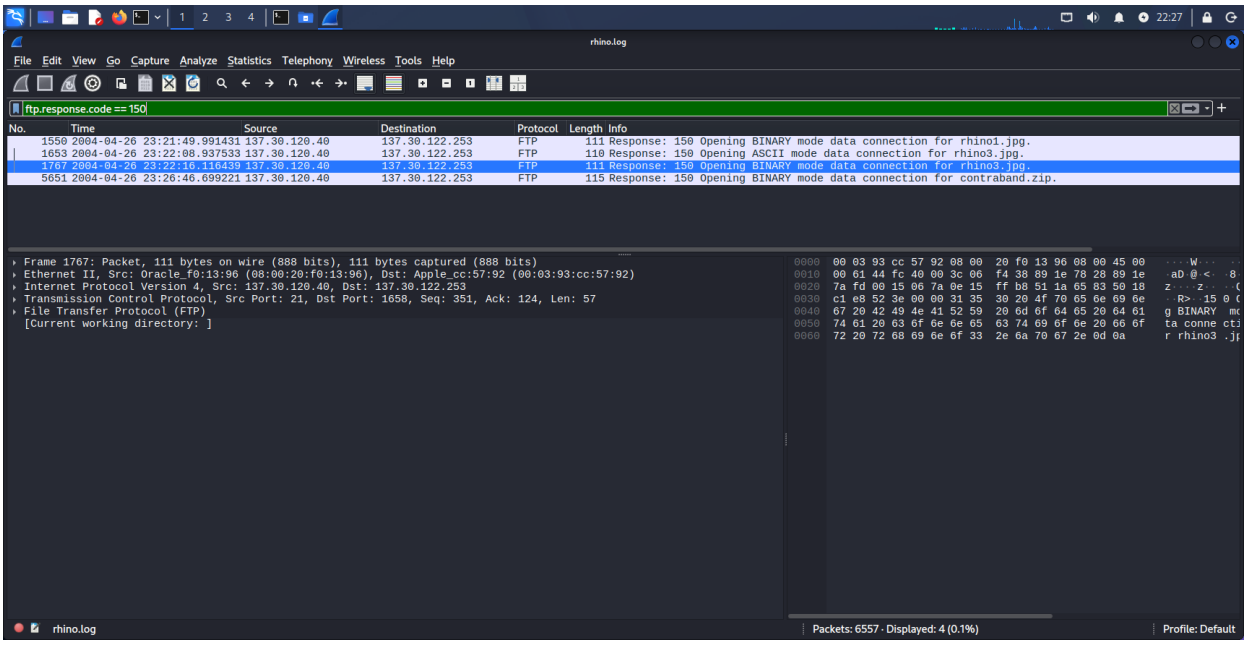
FTP uploads in rhino.log, 2004-04-26 (UTC)

Command frame	Command time	File	150 responses.
1546	22:21:49.954	rhino1.jpg	1550, 22:21:49.991, BINARY
1649	22:22:08.906	rhino3.jpg	1653, 22:22:08.938, ASCII
1763	22:22:16.091	rhino3.jpg	1767, 22:22:16.116, BINARY
5647	22:26:46.672	contraband.zip	5651, 22:26:46.699, BINARY

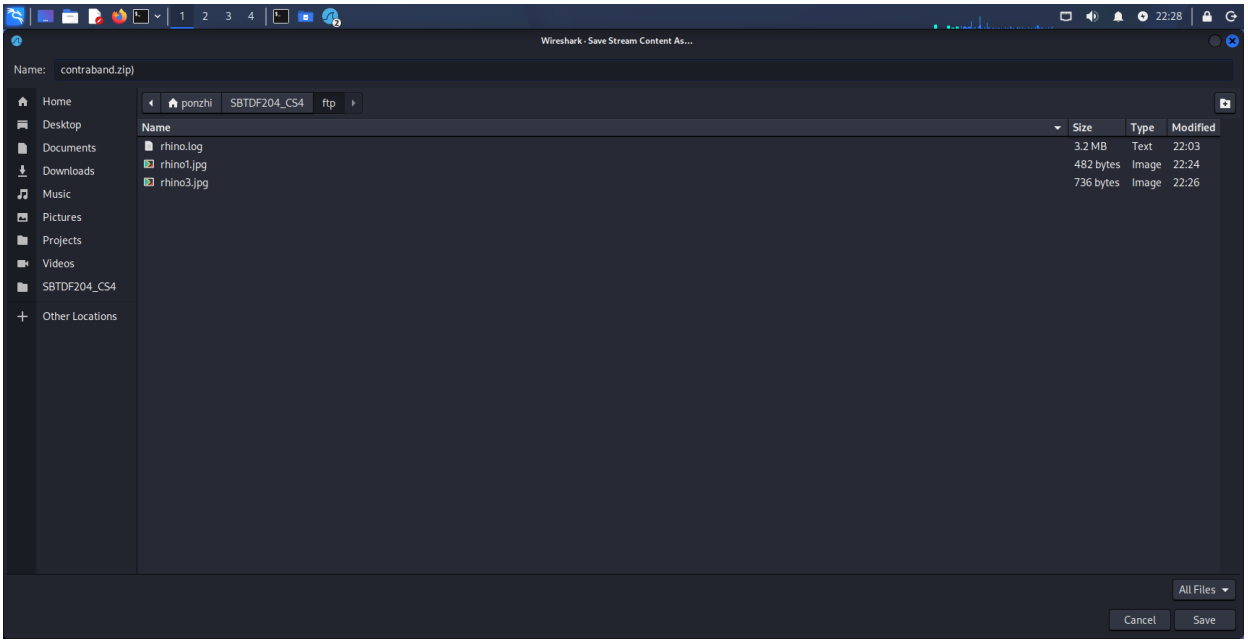
My first attempts to save rhino1.jpg and rhino3.jpg did not work. The files are not images; they are ASCII text. The first part of the text shows "220," which is the beginning of the FTP greeting. I mistakenly followed the control connection instead of the data connection.
I saved contraband.zip from the data connection that started after frame 5651. The saved file is 230,566 bytes and passed the checks.



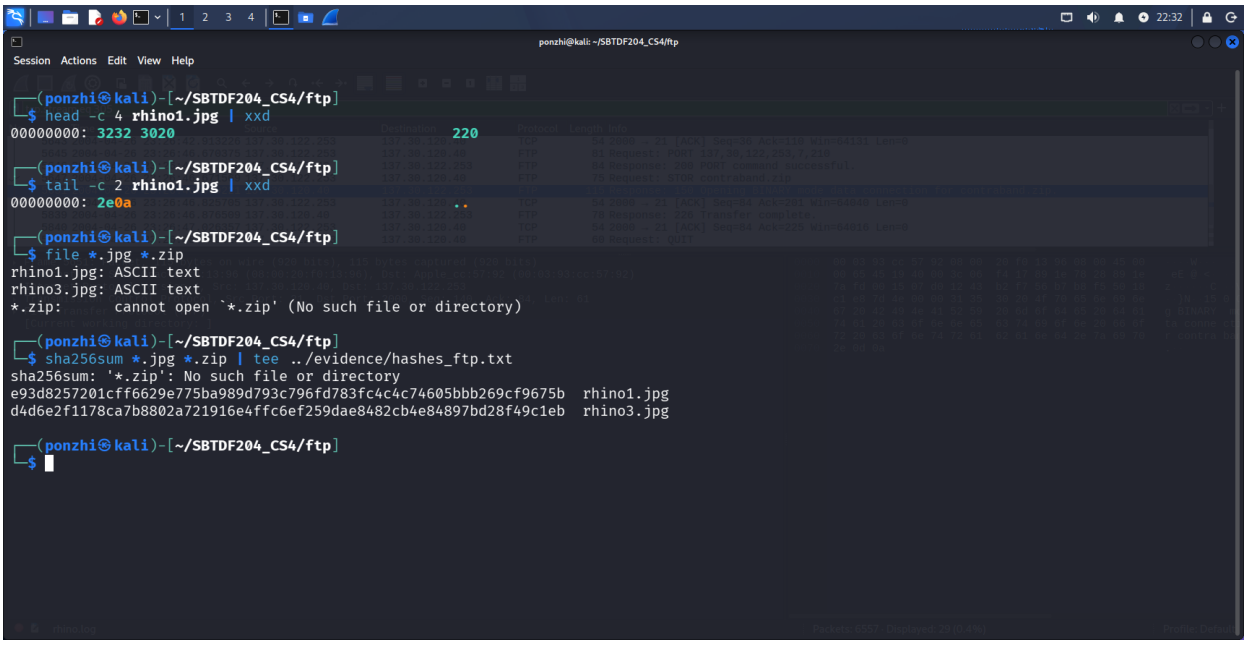
rhino.log in Wireshark filtered for STOR and RETR: four STOR commands.



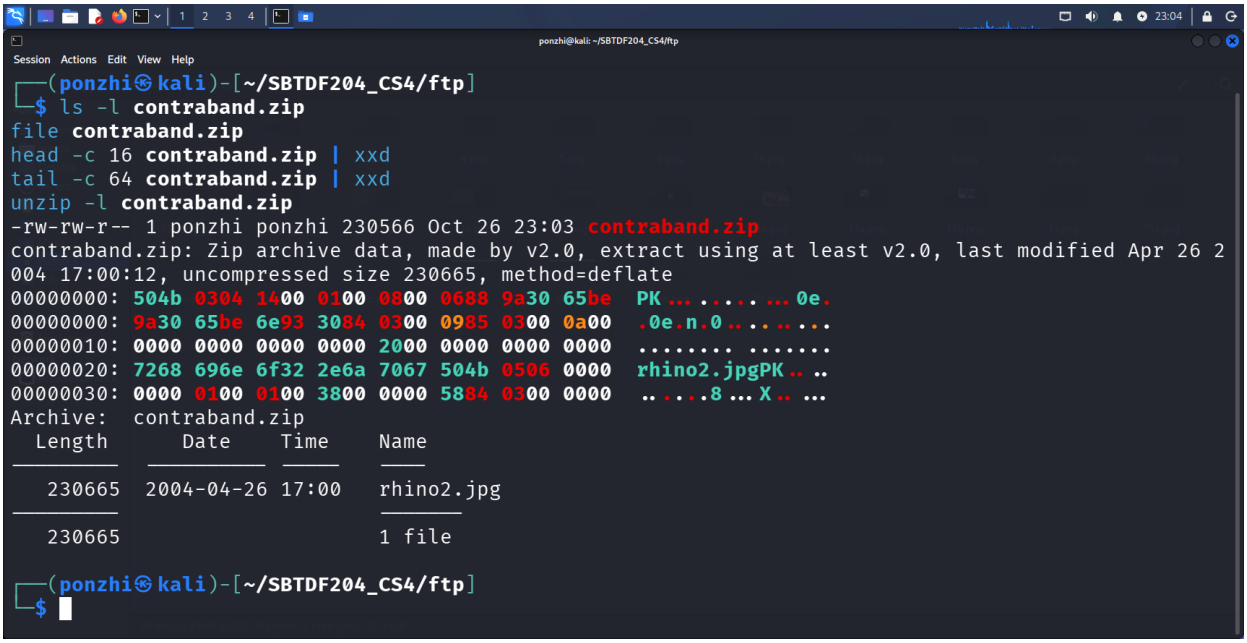
rhino.log filtered for 150 responses: three BINARY and one ASCII data connection.



. Save dialog in ftp/ listing rhino1.jpg and rhino3.jpg



First rhino1.jpg save: ASCII text that begins with the characters "220 ", so the control stream was saved. (E18)



contraband.zip after saving the data connection: size, file type, first and last bytes, and unzip -l. (E19)

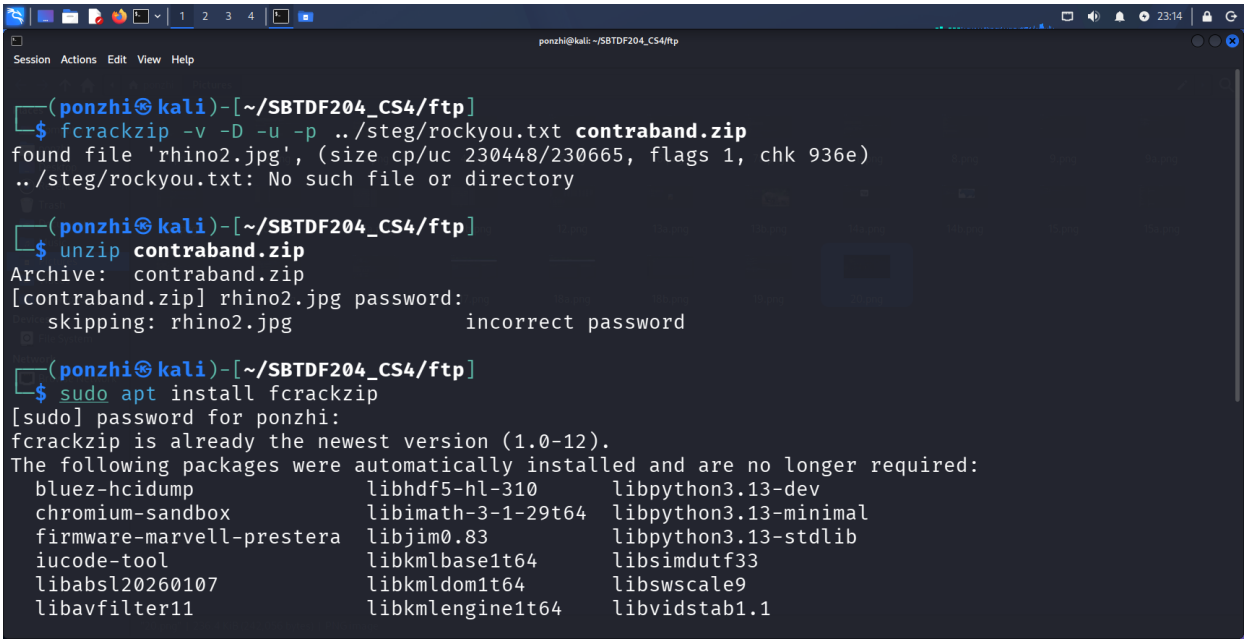
Question 6. What does the protected archive contain, and how was it opened?

File Details: The saved file begins with a local file header signature (50 4b 03 04) and ends with an end-of-central-directory record (50 4b 05 06). This record indicates one entry and a central directory of 56 bytes at offset 230,488. The local entry has 30 header bytes, a 10-byte name, and 230,448 compressed bytes, finishing exactly at offset 230,488. Adding the central directory and the end record (56 and 22 bytes) gives a total file size of 230,566 bytes. When using the command `unzip -l`, it lists one member: rhino2.jpg, which is 230,665 bytes and dated April 26, 2004.

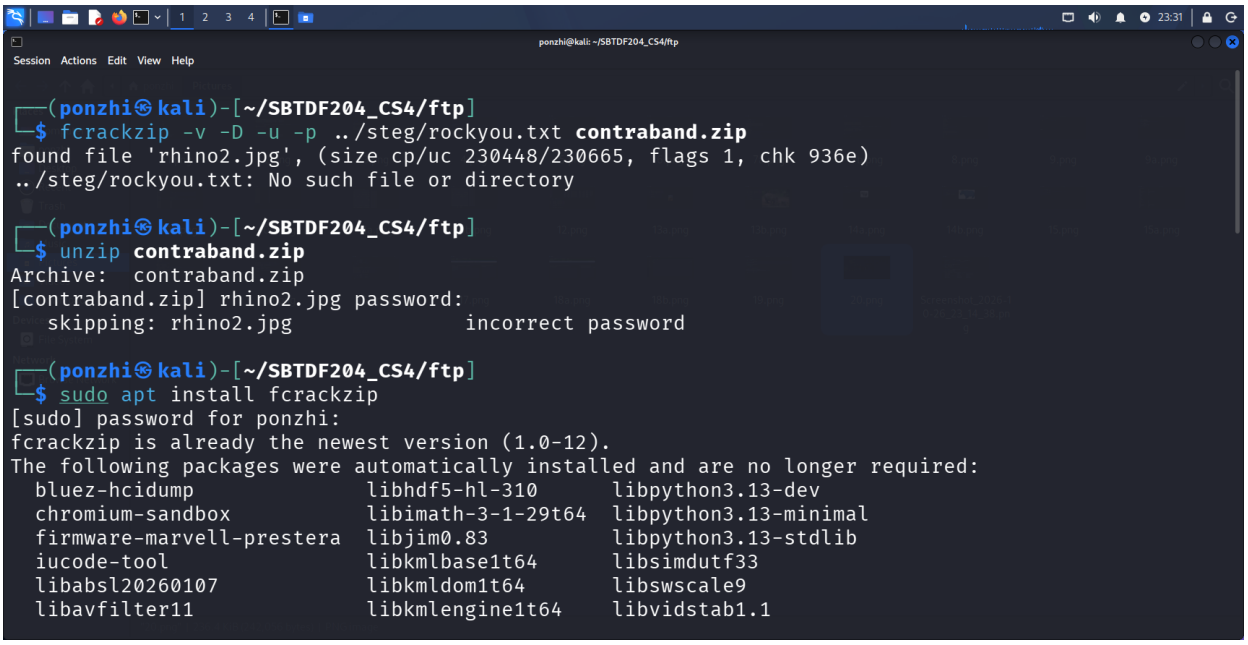
Encryption: In the local header, the general purpose flag is 0x0001. The tool fcrackzip -v shows flag 1 with compressed and uncompressed sizes of 230,448 and 230,665 bytes. When I tried to unzip the file without a password, it asked for one and stated “incorrect password.”

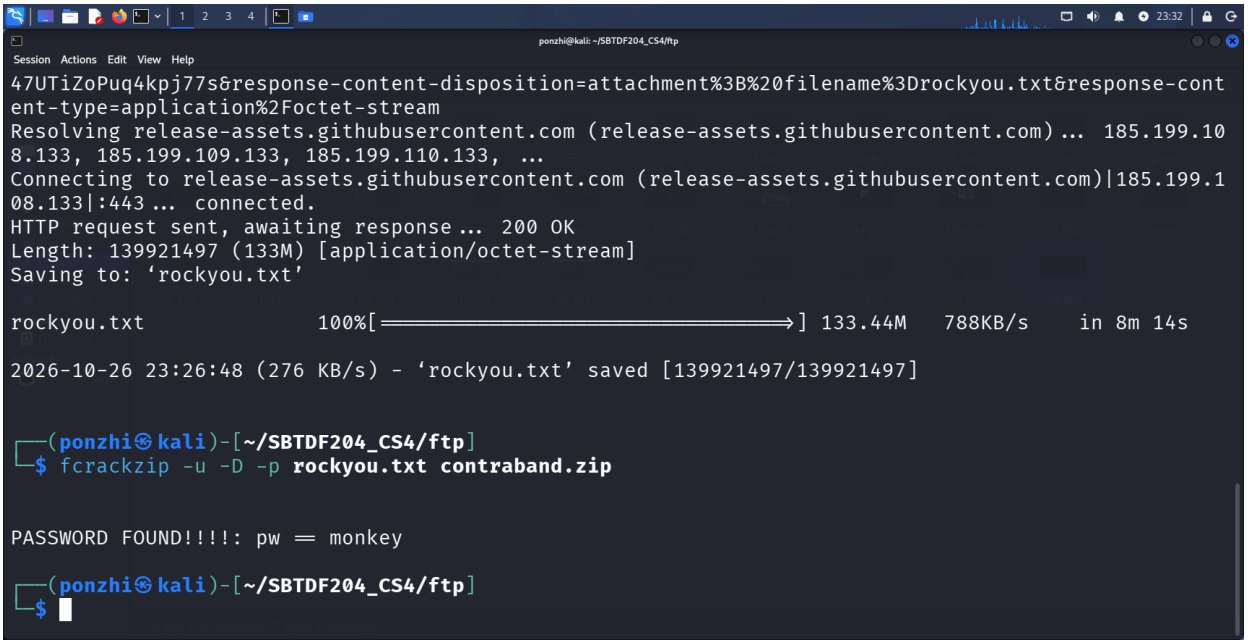
Password Recovery: My first attempt with fcrackzip did not work because the path to the wordlist, ../steg/rockyou.txt, was missing. After downloading rockyou.txt (which is 139,921,497 bytes), I ran the command `fcrackzip -u -D -p rockyou.txt contraband.zip`, and it provided a password that was a common dictionary word. Using that password with unzip, I successfully extracted rhino2.jpg, which is 230,665 bytes and has the same date as the archive from April 2004. I only used this password on this archive.

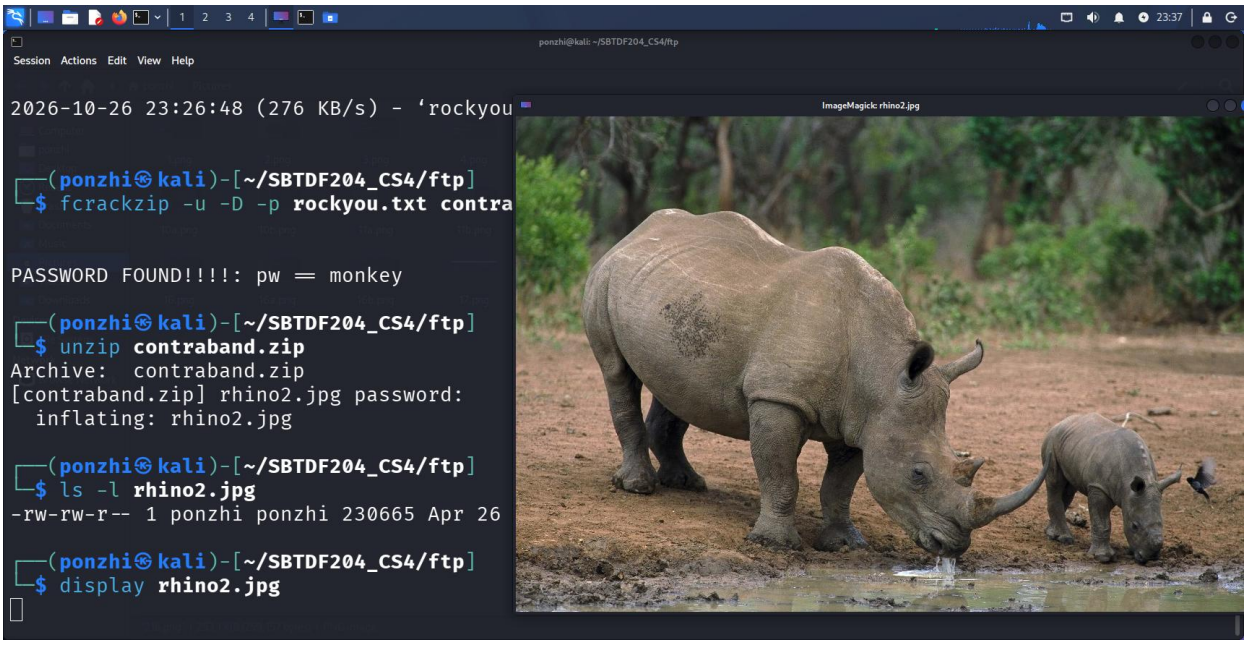
Image Details: The extracted image shows a rhino with a calf at a water hole. Another file, f0106409.jpg, which was carved from the USB image, has the same length (230,665 bytes) and displays the same picture. The two files are likely identical, but I did not capture a hash comparison. Therefore, I can only state that they are equal in length and have matching appearances.



The first FrackZip attempt stops on a wordlist path that does not exist; unzip reports an incorrect password.


ponzhi@kali: ~/SBTDF204_CS4/ftp
Session Actions Edit View Help
┌──(ponzhi㉿kali)-[~/SBTDF204_CS4/ftp]
└─$ fcrackzip -v -D -u -p ../steg/rockyou.txt contraband.zip
found file 'rhino2.jpg', (size cp/uc 230448/230665, flags 1, chk 936e)
../steg/rockyou.txt: No such file or directory
┌──(ponzhi㉿kali)-[~/SBTDF204_CS4/ftp]
└─$ unzip contraband.zip
Archive: contraband.zip
[contraband.zip] rhino2.jpg password:
skipping: rhino2.jpg incorrect password
┌──(ponzhi㉿kali)-[~/SBTDF204_CS4/ftp]
└─$ sudo apt install fcrackzip
[sudo] password for ponzhi:
fcrackzip is already the newest version (1.0-12).
The following packages were automatically installed and are no longer required:
bluez-hcidump libhdf5-hl-310 libpython3.13-dev
chromium-sandbox libimath-3-1-29t64 libpython3.13-minimal
firmware-marvell-prestera libjim0.83 libpython3.13-stdlib
iucode-tool libkmlbase1t64 libsimdutf33
libabsl20260107 libkmldom1t64 libswscale9
libavfilter11 libkmlengine1t64 libvidstab1.1


ponzhi@kali: ~/SBTDF204_CS4/ftp
Session Actions Edit View Help
47UTiZoPuq4kpj77s&response-content-disposition=attachment%3B%20filename%3Drockyou.txt&response-content-type=application%2Foctet-stream
Resolving release-assets.githubusercontent.com (release-assets.githubusercontent.com)... 185.199.108.133, 185.199.109.133, 185.199.110.133, ...
Connecting to release-assets.githubusercontent.com (release-assets.githubusercontent.com)|185.199.108.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 139921497 (133M) [application/octet-stream]
Saving to: 'rockyou.txt'
rockyou.txt 100%[==========>] 133.44M 788KB/s in 8m 14s
2026-10-26 23:26:48 (276 KB/s) - 'rockyou.txt' saved [139921497/139921497]
┌──(ponzhi㉿kali)-[~/SBTDF204_CS4/ftp]
└─$ fcrackzip -u -D -p rockyou.txt contraband.zip
PASSWORD FOUND!!!!: pw == monkey
┌──(ponzhi㉿kali)-[~/SBTDF204_CS4/ftp]
└─$



unzip extracts rhino2.jpg and the picture is displayed. (E22)

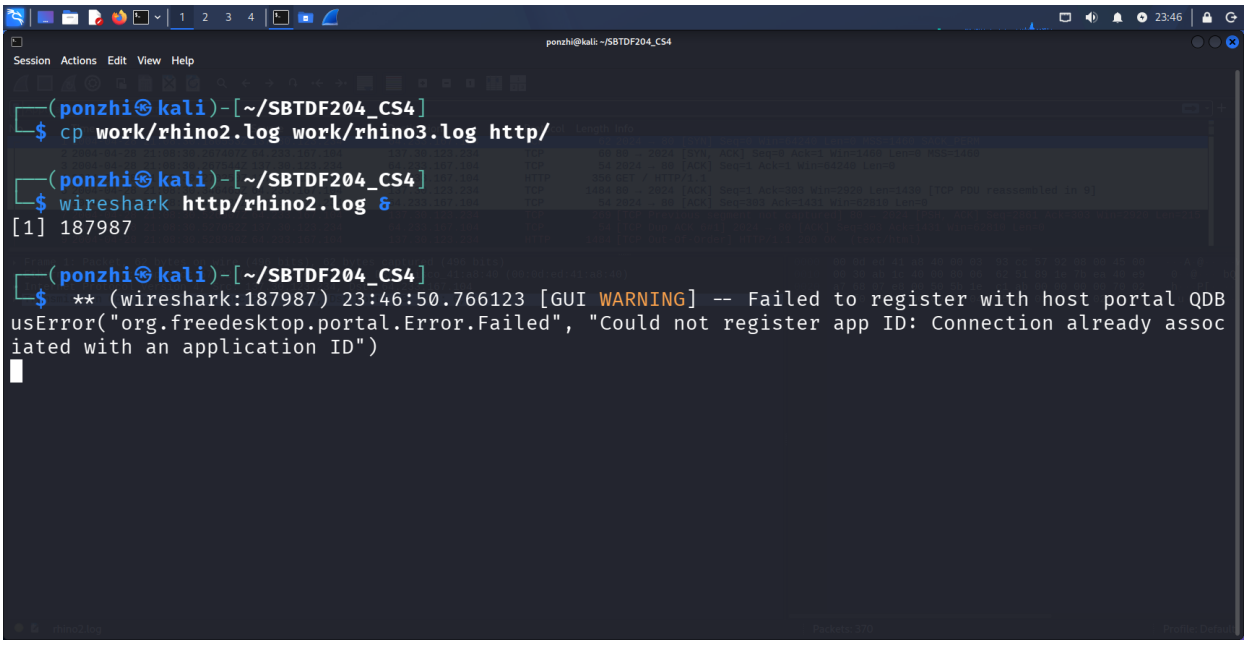
Question 7. What does the HTTP traffic show?

rhino2.log holds 370 packets. It opens with a TCP handshake from 137.30.123.234 port 2024 to 64.233.167.104 port 80 at frame 1 (2004-04-28 21:08:30.181 UTC), a GET / at frame 4 and a 200 OK with content type text/html at frame 9 (Figures 35 and 36, E23). In frames 7 to 9, Wireshark flags a previous segment not captured, a duplicate acknowledgement and an out-of-order segment. The client's Ethernet source address is 00:03:93:cc:57:92 (Apple_cc:57:92).

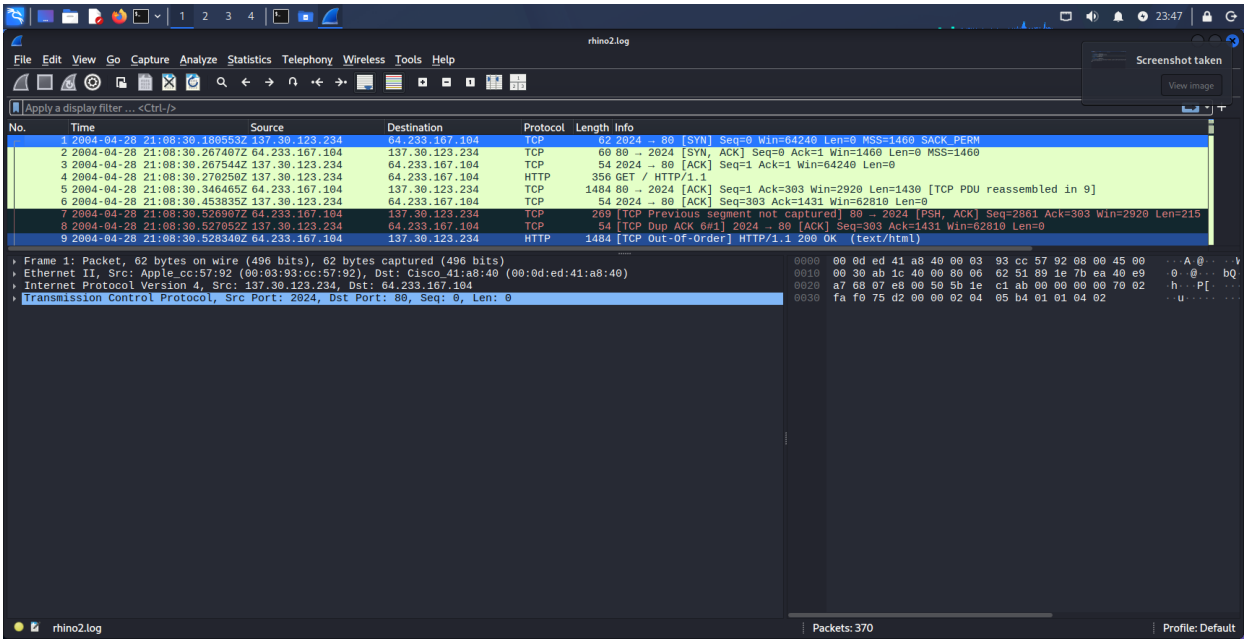
The filter http.request.method == "GET" shows 13 requests listed in Table below.

GET requests in rhino2.log, 2004-04-28 (UTC)

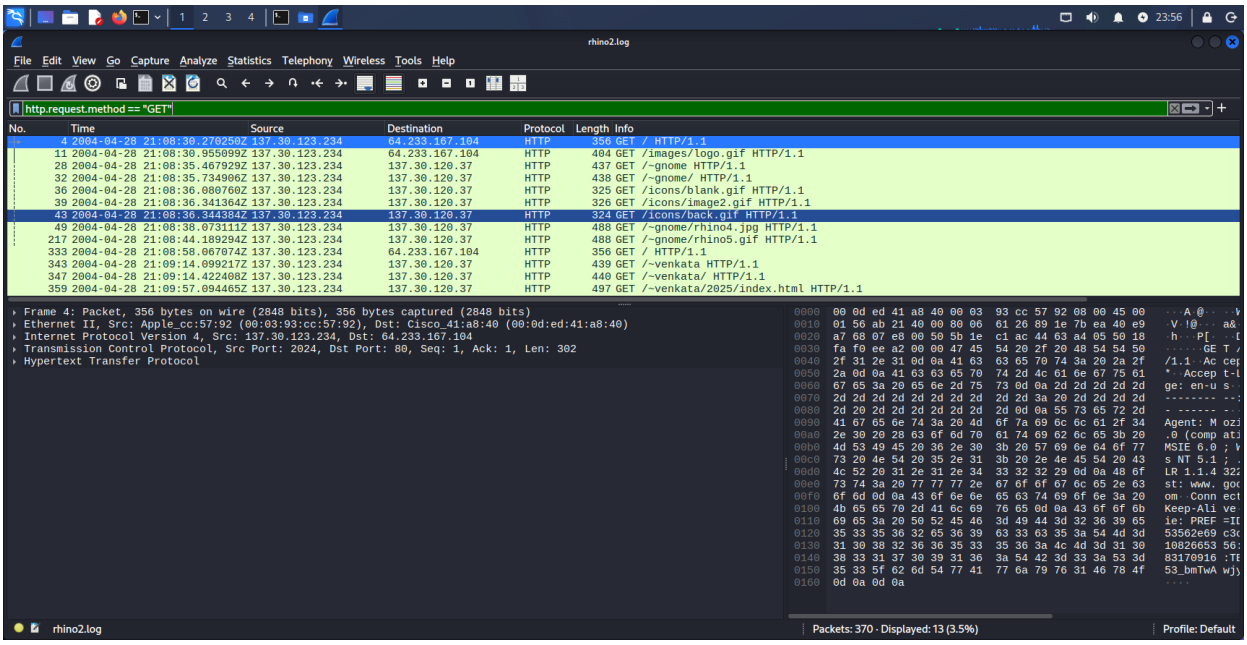
Frame	Time	Server	Request
4	21:08:30.270	64.233.167.104	GET /
11	21:08:30.955	64.233.167.104	GET /images/logo.gif
28	21:08:35.468	137.30.120.37	GET /~gnome
32	21:08:35.735	137.30.120.37	GET /~gnome/
36	21:08:36.081	137.30.120.37	GET /icons/blank.gif
39	21:08:36.341	137.30.120.37	GET /icons/image2.gif
43	21:08:36.344	137.30.120.37	GET /icons/back.gif
49	21:08:38.073	137.30.120.37	GET /~gnome/rhino4.jpg
217	21:08:44.189	137.30.120.37	GET /~gnome/rhino5.gif
333	21:08:58.067	64.233.167.104	GET /
343	21:09:14.099	137.30.120.37	GET /~venkata
347	21:09:14.422	137.30.120.37	GET /~venkata/
359	21:09:57.094	137.30.120.37	GET /~venkata/2025/index.html



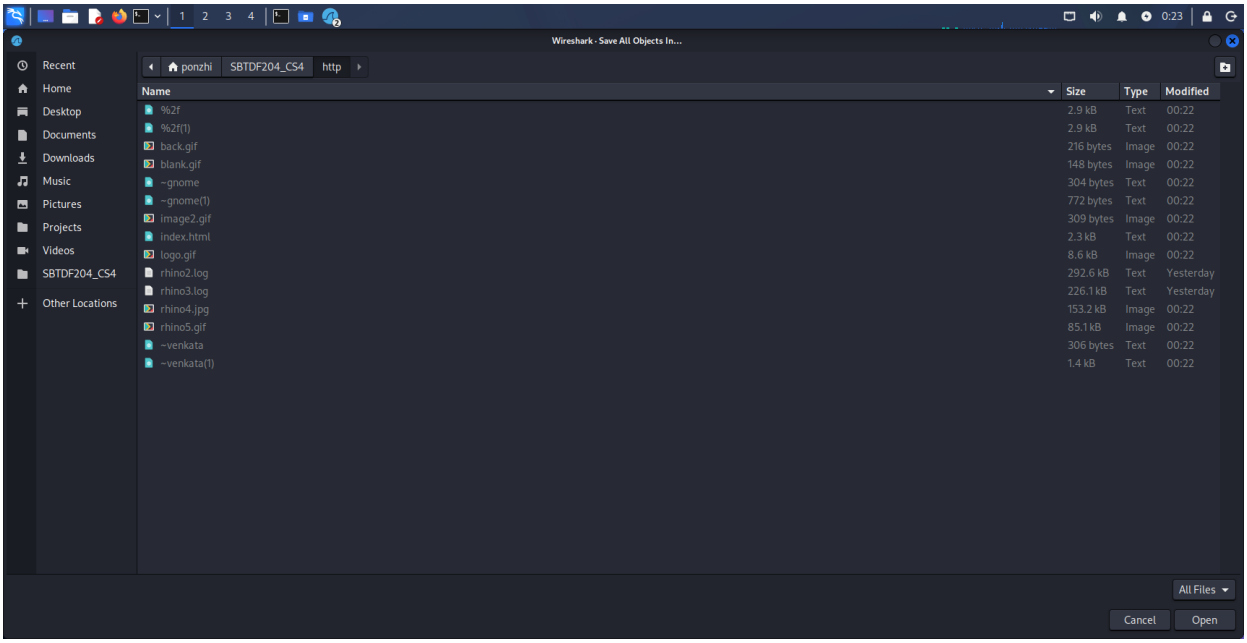
rhino2.log and rhino3.log copied to http/ and rhino2.log opened in Wireshark.



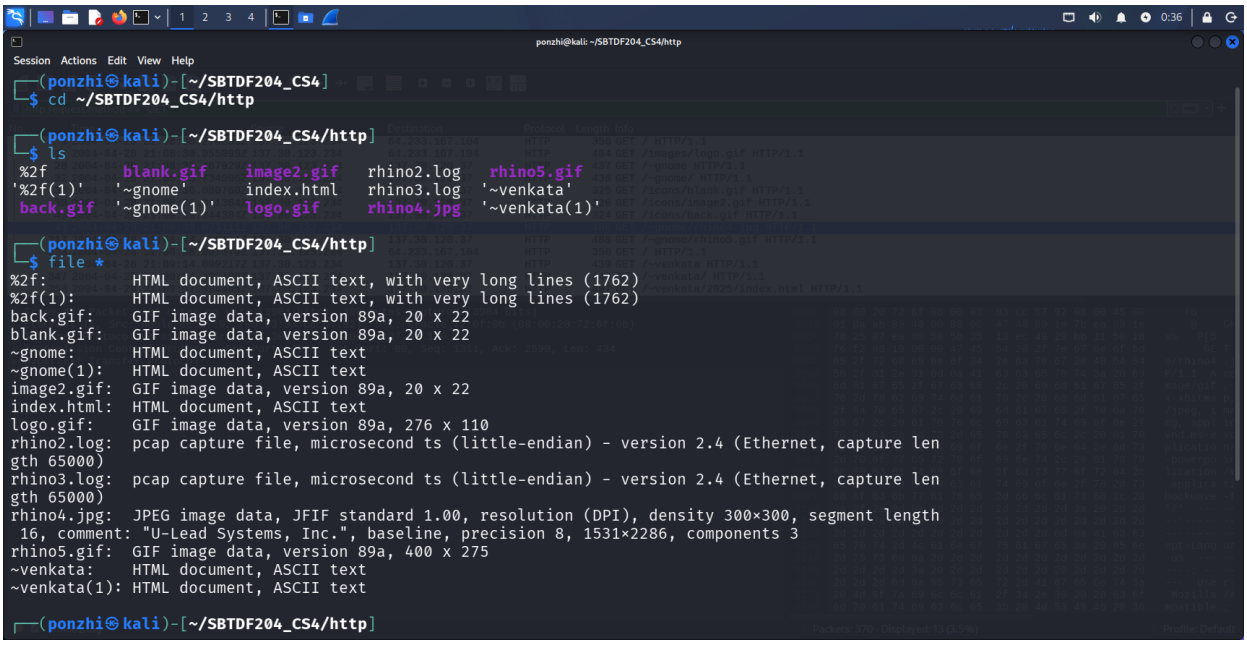
Opening frames of rhino2.log: TCP handshake, GET /, first 200 OK and flagged segments.



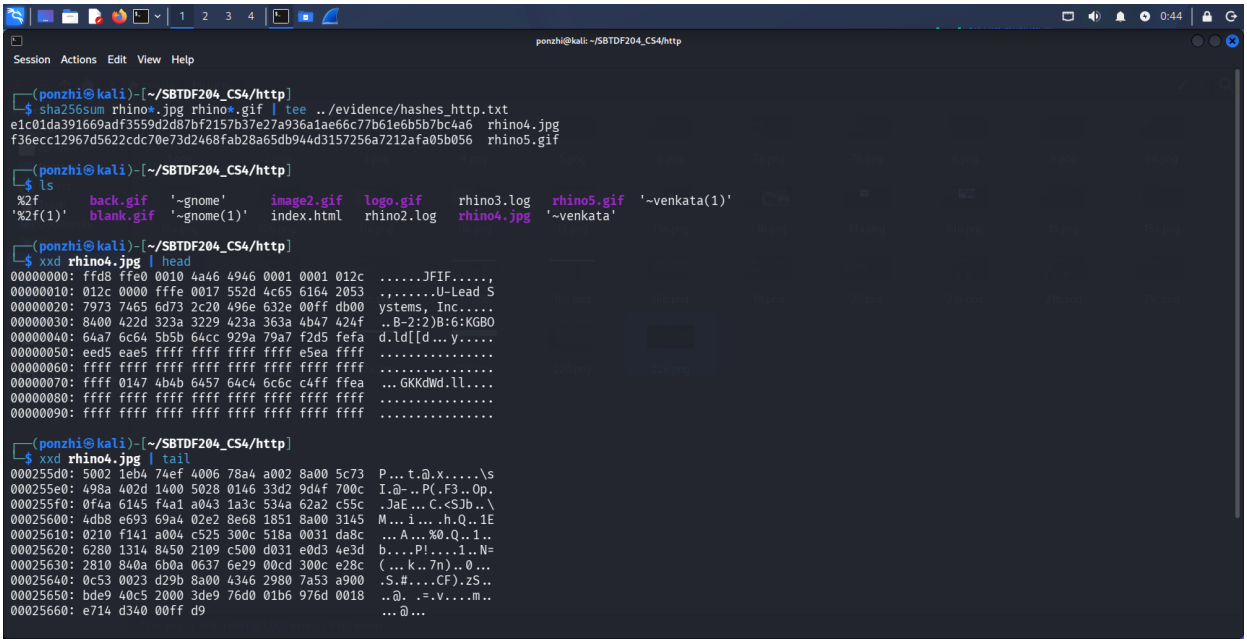
rhino2.log filtered for http.request.method == "GET": 13 of 370 packets.



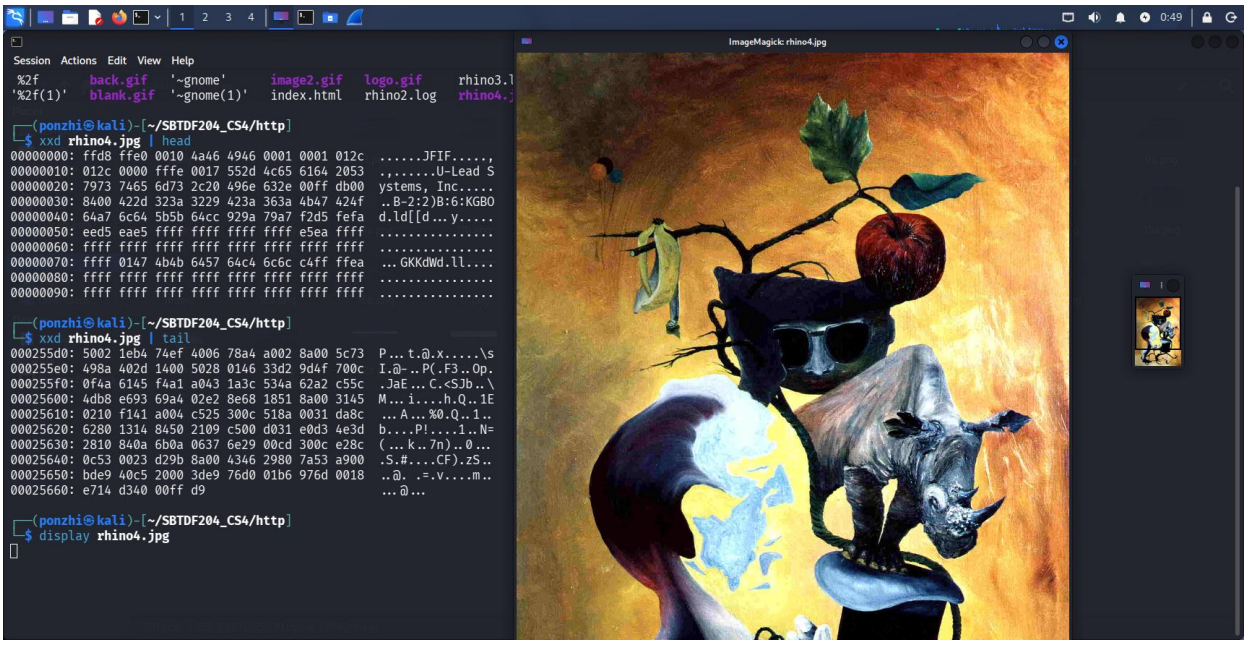
Export Objects, Save All: thirteen objects saved into http/.



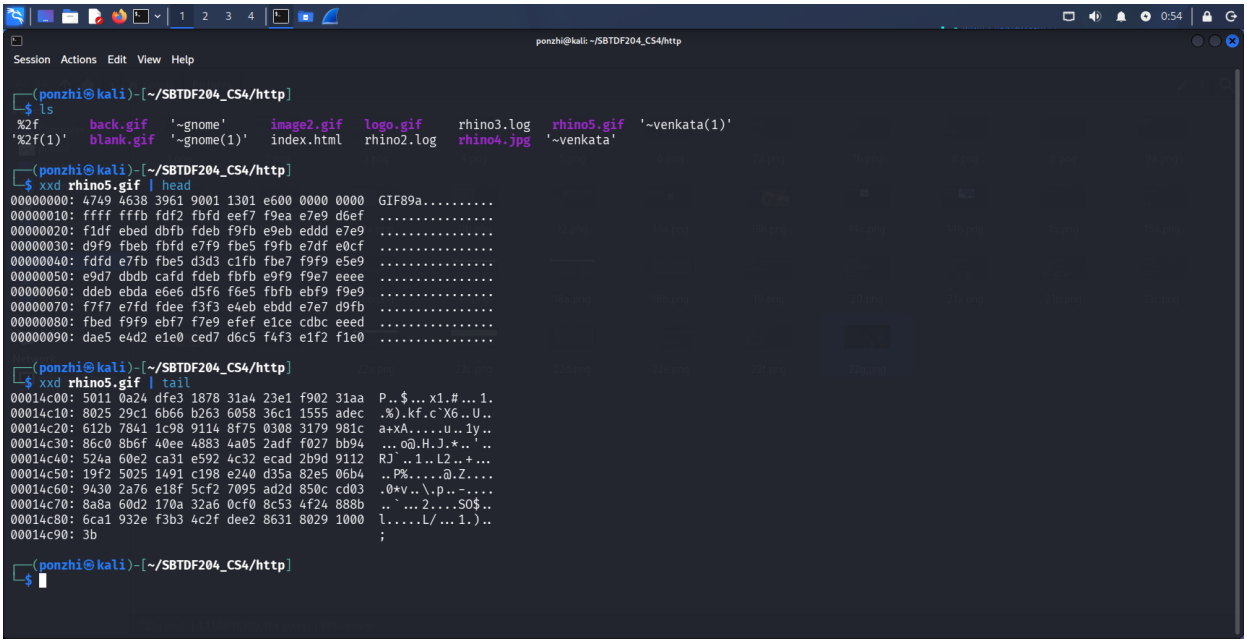
file * in http/: JPEG, GIF, HTML and capture-file identifications.



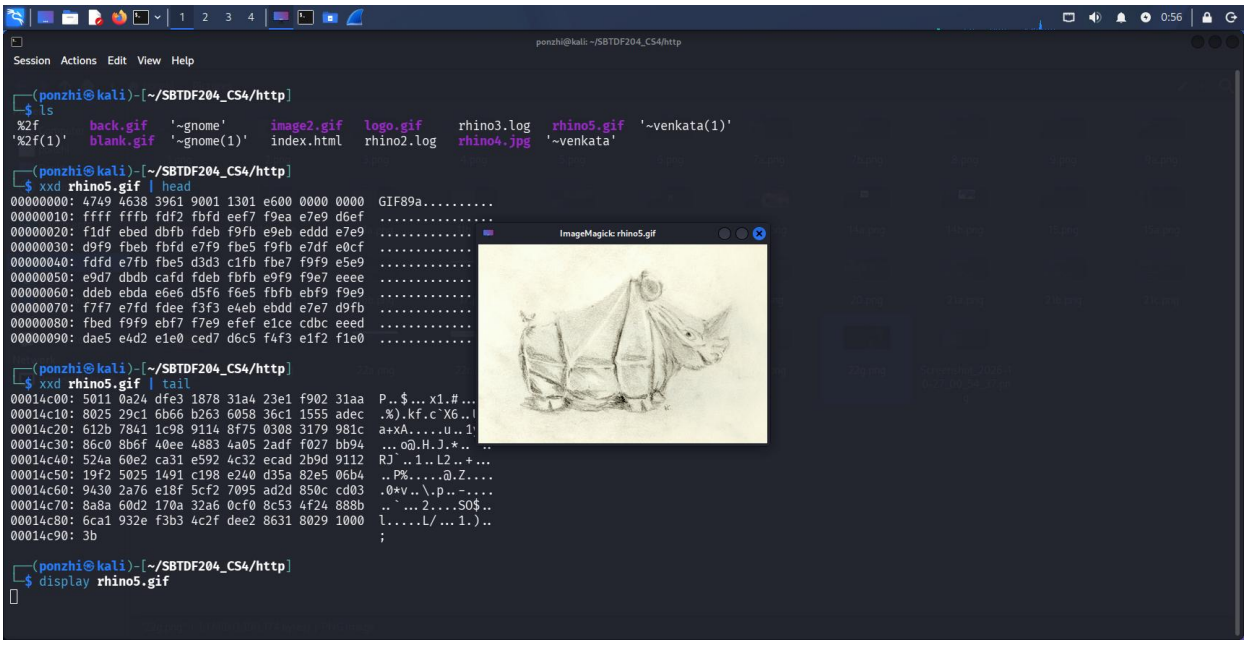
SHA-256 of rhino4.jpg and rhino5.gif, and first and last bytes of rhino4.jpg.



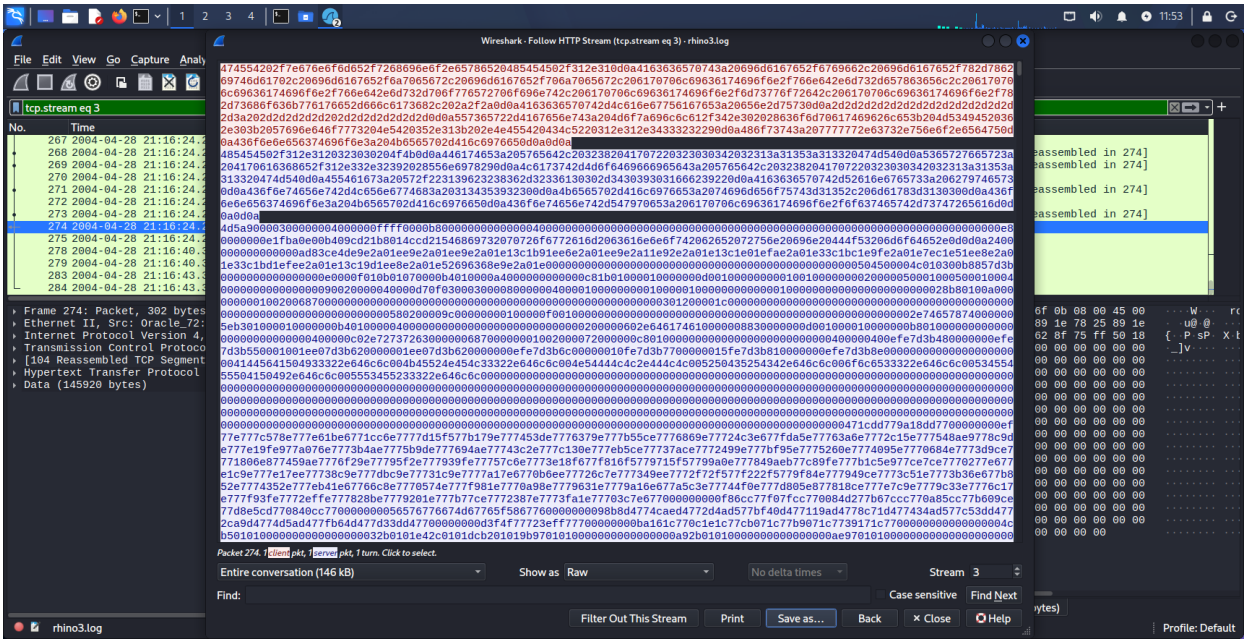
rhino4.jpg first and last bytes, and the image displayed.



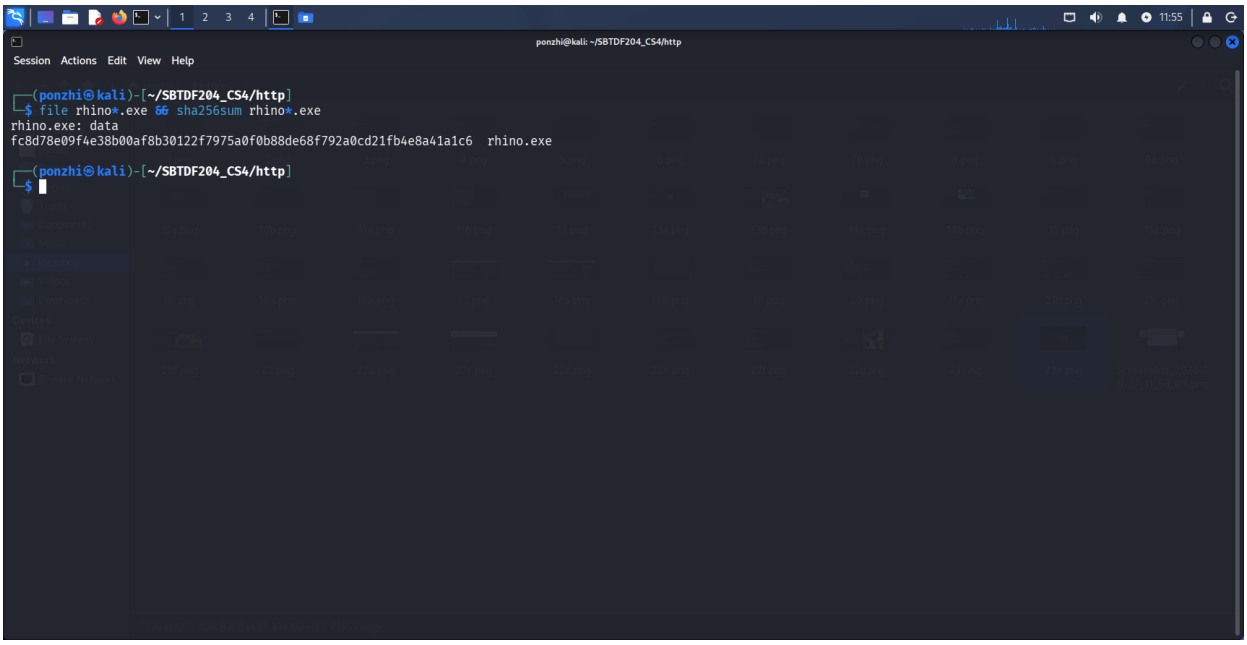
rhino5.gif



rhino5.gif displayed.



rhino3.log, Follow HTTP Stream on tcp.stream eq 3, Show data as Raw.



File and sha256sum of the saved rhino.exe. file reports data.

Question 8. What conclusion can I defend?

I observed the following in the files:

- RHINOUSB.dd is a FAT16 drive with two recipe files and no deleted entries. It has nine carved image files, eight of which are unique based on SHA-256.
- The file rhino.log shows four FTP uploads from IP address 137.30.122.253 to 137.30.120.40 on April 26, 2004, between 22:21:49 and 22:26:46 UTC.
- The file contraband.zip is a complete, encrypted archive containing one image, rhino2.jpg.
- The file rhino2.log lists requests for rhino4.jpg and rhino5.gif from IP address 137.30.123.234 to 137.30.120.37 on April 28, 2004, at 21:08 UTC. The file rhino3.log has a response that includes an executable around 21:16 UTC.
- The Ethernet address 00:03:93:cc:57:92 appears in frame 1546 of rhino.log and frame 4 of rhino2.log.

From this information, I can say:

- The client uploaded contraband.zip and likely also sent the picture inside it to the server. This is based on the STOR command and the complete archive.
- Both captures came from the same network card because the Ethernet address matches. The IP addresses 137.30.122.253 and 137.30.123.234 are different, indicating the client changed their IP address between the two captures.
- The image rhino2.jpg in the archive is probably the same as f0106409.jpg on the USB drive since their lengths and images match. A hash comparison would confirm this.

Timeline and correlation

This table outlines the stages of the course template in chronological order. All times are in UTC. The sequence is as follows:

- FTP uploads occurred on April 26, 2004, from 22:21:49 to 22:26:46.
- HTTP requests took place on April 28, 2004, from 21:08:30 to 21:09:57.
- The executable response happened at about 21:16:24 on the same day.

Timeline of material events (UTC)

Stage	UTC timestamp	ID	Source artefact	Observed fact	Interpretation and limit
Media preservation	2026-10-26 about 16:51	E01, E02	original/ and work/	SHA-256 of four originals recorded, originals set read-only, copies hash identically	Clock read from the Kali taskbar, 17:51 at UTC+1. Examination event.
Media preservation	2026-10-26 19:37:54	E03	date -u	Reference UTC time and tool versions recorded	Fixes the examiner clock.
Disk image review	No dated event	E04 to E06	RHINOUSB.dd, FAT16	Two allocated files, no deleted entries, no partition table	FAT timestamps were not captured.
Recovered or hidden content	2026-10-26 about 20:17	E09	PhotoRec on the image copy	132 files carved, nine image files, eight unique	Clock read from the taskbar, 21:17 at UTC+1. Examination event.
FTP transfer activity	2004-04-26 22:21:49.954	E16, E17	rhino.log frames 1546, 1550	STOR rhino1.jpg, 137.30.122.253 to 137.30.120.40:21. BINARY data connection 37 ms later.	Upload started. Object not reconstructed.
FTP transfer activity	2004-04-26 22:22:08.906	E16, E17	rhino.log frames 1649, 1653	STOR rhino3.jpg, response 150 in ASCII mode at 22:22:08.938	ASCII mode can alter binary data. Not reconstructed.
FTP transfer activity	2004-04-26 22:22:16.091	E16, E17	rhino.log frames 1763, 1767	STOR rhino3.jpg, response 150 in BINARY mode at 22:22:16.116	About seven seconds after the ASCII attempt. Consistent with a retry (inference).
FTP transfer activity	2004-04-26 22:26:46.672	E16, E17, E19	rhino.log frames 5647, 5651	STOR contraband.zip, BINARY data connection at 22:26:46.699. Saved archive is complete and holds rhino2.jpg.	Records the transfer, not who ran it.

Stage	UTC timestamp	ID	Source artefact	Observed fact	Interpretation and limit
HTTP activity	2004-04-28 21:08:30.181	E23	rhino2.log frames 1, 4	TCP SYN from 137.30.123.234:2024 to 64.233.167.104:80, then GET / at 21:08:30.270	46 hours 41 minutes after the contraband.zip upload. Start of the capture.
HTTP activity	2004-04-28 21:08:38.073	E24, E27	rhino2.log frame 49	GET /~gnome/rhino4.jpg to 137.30.120.37	Image requested and extracted.
HTTP activity	2004-04-28 21:08:44.189	E24, E27	rhino2.log frame 217	GET /~gnome/rhino5.gif to 137.30.120.37	Image requested and extracted.
HTTP activity	2004-04-28 about 21:16:24	E30, E31	rhino3.log stream 3, frame 274	GET /~gnome/rhino.exe, 200 OK, 145,920 byte body starting 4d 5a	Executable response recorded and not run. Time as displayed with UTC display set.
Corroborated conclusion	2004-04-26 22:21 to 2004-04-28 21:16	E16 to E31	rhino.log frame 1546 and rhino2.log frame 4	Same Ethernet source address 00:03:93:cc:57:92 in both captures. Client IP differs.	Points to one network card, not one person. The address can be changed.

The upload at frame 5647 sends a file named contraband.zip, which contains an image called rhino2.jpg. The USB image has a file named f0106409.jpg, and both files are the same size and show the same picture. If a hash check proves they are the same, then both the USB and the upload share one file. However, we still do not know the order of events or the person involved.

In the case of FTP and HTTP, both frame 1546 (FTP) and frame 4 (HTTP) show the Ethernet source address 00:03:93:cc:57:92. Wireshark identifies this as Apple_cc:57:92. The client IP address is 137.30.122.253 in the first capture and 137.30.123.234 in the second, which is a different address on the same network. This suggests that one network card switched between addresses during the 46 hours between the captures. The address might also have been changed by software.

For HTTP and USB, the sizes of rhino4.jpg (153,191 bytes) and rhino5.gif (85,137 bytes) do not match any files recovered from the USB image. The upload of rhino1.jpg was about 65.7 kB, and no recovered file is that size either. These files do not support each other based on their sizes.

Conclusion

The evidence supports a clear account of the client's actions on the network. They uploaded files to an FTP server on April 26, 2004, and made web requests and a download on April 28, 2004. The USB image also shows that there is image data in unallocated space. Table 7 explains how each claim is supported.

Propositions and confidence

Proposition	Support	Confidence
The client at 137.30.122.253 uploaded contraband.zip to 137.30.120.40 at about 22:26:46 UTC on 2004-04-26	STOR and 150 frames, complete archive that opens with the recovered password	High, as a statement about the capture
The archive holds an image that also appears on the USB image	Equal length, same picture, no hash comparison	Moderate, pending a hash comparison
One network card produced the FTP and the HTTP traffic	Same Ethernet address in both captures, different IP addresses	Moderate
The machine that produced the traffic is linked to the USB device	Indirect, through one matching file	Low to moderate
The USB image holds hidden or embedded content	Detection tool would not build	Not assessed
A named person possessed or sent the images	No user-identifying artefact examined	None

Appendices Evidence log

ID	Tool and method	Source record, stream, or path	Observed fact
E01	stat, sha256sum	original/RHINOUSB.dd, rhino.log, rhino2.log, rhino3.log	Sizes 259,506,176; 3,187,907; 292,604; 226,094 bytes. SHA-256 recorded for all four (Table 1).
E02	chmod 444, cp, sha256sum	work/	Working copies hash identically to the originals.
E03	date -u and version commands	Kali session	2026-10-26 19:37:54 UTC. Sleuth Kit 4.14.0, PhotoRec 7.2, TShark 4.6.9, ExifTool 13.55.
E04	file, fdisk -l, mmls	work/RHINOUSB.dd	DOS boot sector, 16-bit FAT, 506,848 sectors, serial 0x4092d9d1. mmls returns no partitions.
E05	fsstat	work/RHINOUSB.dd	FAT16. FATs at sectors 1 to 248 and 249 to 496, root 497 to 528, clusters 529 to 506840, 4,096-byte clusters.
E06	fls -r, fls -rd	work/RHINOUSB.dd	Allocated gumbo1.txt (inode 4) and gumbo2.txt (inode 6). No deleted or orphan entries.
E07	icat	Inodes 4 and 6	Two cooking recipes, unrelated to the allegation.
E08	cp, sha256sum	work/RHINOUSB_photorec.dd	Copy hash ce550424...6a32be65 equals the original.
E09	PhotoRec 7.2	RHINOUSB_photorec.dd to recovered/recup_dir.1	One FAT16 partition, 132 files saved.
E10	file	recovered/recup_dir.1/*	Seven JPEG, two GIF, one Word document, report.xml, text fragments.
E11	sha256sum	recovered/recup_dir.1/*	f0105873.jpg and f0335081.jpg share SHA-256 f92654d9...447596fb. Eight unique images among nine files.
E12	ls -l	recup_dir.1	JPEG sizes 95,814 to 415,534 bytes. GIF sizes 11,407 and 4,105 bytes.

ID	Tool and method	Source record, stream, or path	Observed fact
E13	display	f0106393.jpg, f0106409.jpg, f0106865.gif, f0106889.gif	Each shows a picture of a rhino.
E14	cp, ExifTool	steg/*.jpg	JFIF fields and comments. No sign of embedded data in the captured output.
E15	autoreconf, configure, make	steg/stegdetect	jpeg-6b compiler test fails. stegdetect and stegbreak not built.
E16	Wireshark, ftp.request.command	rhino.log frames 1546, 1649, 1763, 5647	Four STOR uploads, 137.30.122.253 to 137.30.120.40:21. Frame 1546: 46 data frames, 65,703 bytes.
E17	Wireshark, ftp.response.code == 150	Frames 1550, 1653, 1767, 5651	Three BINARY and one ASCII data connection.
E18	Follow TCP Stream, Save as, xxd, file	ftp/rhino1.jpg, ftp/rhino3.jpg	482 and 736 byte ASCII text files starting "220 ". Control stream, not images.
E19	Follow TCP Stream, xxd, file, unzip -l	ftp/contraband.zip	230,566 bytes. Zip signature and end record. One member, rhino2.jpg, 230,665 bytes. Encrypted flag set.
E20	fcrackzip -v, unzip	ftp/contraband.zip	Flags 1, sizes 230,448 and 230,665. unzip reports incorrect password. First wordlist path missing.
E21	wget, fcrackzip -u -D -p	ftp/rockyou.txt	Wordlist of 139,921,497 bytes. Password found, course lab only.
E22	unzip, ls -l, display	ftp/rhino2.jpg	Extracted at 230,665 bytes. Shows a rhino with a calf at water.
E23	Wireshark	http/rhino2.log frames 1 to 9	370 packets. Client 137.30.123.234:2024 to 64.233.167.104:80. Flagged segments in frames 7 to 9.
E24	Wireshark, http.request.method	rhino2.log	13 GET requests. Frame 49 rhino4.jpg and frame 217 rhino5.gif to 137.30.120.37.
E25	Export Objects, Save All	http/	13 objects saved, including rhino4.jpg 153.2 kB and rhino5.gif 85.1 kB.

ID	Tool and method	Source record, stream, or path	Observed fact
E26	file	http/*	rhino4.jpg JPEG 1531 x 2286. rhino5.gif GIF89a 400 x 275. HTML pages and icons.
E27	sha256sum, xxd	http/rhino4.jpg, http/rhino5.gif	SHA-256 e1c01da3...5b7bc4a6 and f36ecc12...afa05b056. rhino4.jpg ends ff d9 at 153,191 bytes.
E28	display	http/rhino4.jpg	A painted scene that includes a rhino.
E29	xxd, display	http/rhino5.gif	GIF89a header, 3b trailer, 85,137 bytes. Pencil sketch of a rhino.
E30	Wireshark, Follow HTTP Stream	rhino3.log tcp.stream eq 3, frame 274	GET /~gnome/rhino.exe, 200 OK, 145,920 byte body in 104 segments, starts 4d 5a.
E31	file, sha256sum	http/rhino.exe	file reports data. SHA-256 fc8d78e0...8a41a1c6 of the saved stream.